Python Installation and Basics

Python Overview

শুরুতেই একটা প্রশ্ন আসতে পারে - এতো এতো প্রোগ্রামিং ল্যাংগুয়েজ থাকতে মেশিন লার্নিং শেখার জন্য Python কেন?

উত্তরঃ

সহজ ও বোধগম্য সিনট্যাক্স পাইথনের কোড অন্যান্য ভাষার তুলনায় অনেক সহজে পড়া ও লেখা যায়।
যারা নতুন, তারাও দ্রুত শিখতে পারে।

লাইব্রেরি ও ফ্রেমওয়ার্ক সমৃদ্ধ

মেশিন লার্নিং ও ডেটা সায়েন্সের জন্য পাইথনে অসংখ্য লাইব্রেরি আছে:

- NumPy, Pandas → ডেটা প্রসেসিং
- Matplotlib, Seaborn → ভিজ্যুয়ালাইজেশন
- Scikit-learn → ক্লাসিক্যাল ML অ্যালগরিদম
- TensorFlow, PyTorch → ডিপ লার্নিং
- NLTK, SpaCy → ন্যাচারাল ল্যাঙ্গুয়েজ প্রসেসিং

আরেকটা কথা বলে রাখা ভালো, শুধু মেশিন লার্নিং নয়, পাইথন এখন শুধু মেশিন লার্নিং-এ না, আধুনিক প্রযুক্তির প্রায় সব বড় ক্ষেত্রেই ব্যবহার হচ্ছে।

মেশিন লার্নিং ও এআই **(Artificial Intelligence)**

- TensorFlow, PyTorch, Scikit-learn দিয়ে মডেল বানানো হয়।
- Self-driving car, Recommendation system (Netflix, YouTube), Fraud detection—সবখানেই পাইথন আছে।

ডেটা সায়েন্স ও অ্যানালিটিক্স

- NumPy, Pandas, Matplotlib, Seaborn দিয়ে ডেটা পরিষ্কার, বিশ্লেষণ, ভিজ্যুয়ালাইজেশন করা হয়।
- কোম্পানির সিদ্ধান্ত নেওয়ার জন্য Data-driven insights বের করা হয় পাইথন দিয়েই।

ওয়েব ডেভেলপমেন্ট

- Django, Flask, FastAPI দিয়ে স্কেলেবল ওয়েব অ্যাপ ও REST API বানানো হয়।
- Instagram, Pinterest এর ব্যাকএন্ডে পাইথন ব্যবহার হয়।

অটোমেশন ও স্ক্রিপ্টিং

- Repetitive কাজ (যেমন Excel প্রসেসিং, ফাইল ম্যানেজমেন্ট, ওয়েব স্ক্র্যাপিং) পাইথন দিয়ে সহজে অটোমেট করা যায়।
- DevOps, QA টেস্টিং-এও পাইথন স্ক্রিপ্ট ব্যবহৃত হয়।

ডিপ লার্নিং ও কম্পিউটার ভিশন

- Face recognition, Object detection, Medical image analysis সবই পাইথন লাইব্রেরি (OpenCV, Keras, PyTorch) দিয়ে হয়।

ন্যাচারাল ল্যাঙ্গুয়েজ প্রসেসিং **(NLP)**

- ChatGPT, Google Translate, Voice assistant (Siri, Alexa)-এর মূল ভিত্তি পাইথন লাইব্রেরি (NLTK, SpaCy, Transformers)।

সাইবারসিকিউরিটি

- নেটওয়ার্ক মনিটরিং, পেন-টেস্টিং, মালওয়্যার অ্যানালাইসিসে পাইথন ব্যবহার হয়।
- টুল যেমন Scapy, PyCrypto ইত্যাদি সাইবারসিকিউরিটিতে কাজে লাগে।

গেম ডেভেলপমেন্ট

- Pygame, Panda3D দিয়ে ছোট বড় গেম বানানো যায়।
- বড় গেম ইঞ্জিনেও পাইথন scripting ল্যাঙ্গুয়েজ হিসেবে ব্যবহৃত হয়।

IoT (Internet of Things)

- Raspberry Pi, Microcontroller প্রোজেক্টে পাইথন ব্যবহার করে সেন্সর কন্ট্রোল, হোম অটোমেশন, রোবটিক্স তৈরি হয়।

ব্লকচেইন ও ফিনটেক

- ক্রিপ্টোকারেন্সি, স্মার্ট কন্ট্রাক্ট টেস্টিং, ট্রেডিং বট তৈরিতেও পাইথন ব্যবহার হয়।

Installing Python and Compiler

মডিউলে দেখানো গাইডলাইন ফলো করে আমাদের ডিভাইসে Python ও VS Code (Code editor) ইন্সটল করে নিই (যদি অলরেডি ইনস্টল করা না থাকে)

প্যাকেজ লিঙ্কসঃ
Python : https://www.python.org/
VS Code: https://code.visualstudio.com/

CMD তে নিচের কমান্ড ব্যবহার করে python installation চেক করে নিই -
python --version

এবার VS Code এ নিচের Extension গুলো ইন্সটল করে নিই-
Python
Jupyter

Variables and Data Type in Python

Variable কী?

Variable হলো এমন একটি নাম, যেখানে আমরা কোনো ডেটা (value) সংরক্ষণ করি।

এটা মূলত কম্পিউটারের মেমোরিতে একটি জায়গার রেফারেন্স।

উদাহরণ:

```
name = "Mehrab"
age = 28
price = 99.99
```

এখানে

- name → একটি স্ট্রিং ভ্যালু সংরক্ষণ করছে
- age → একটি integer ভ্যালু সংরক্ষণ করছে
- price → একটি floating-point সংখ্যা সংরক্ষণ করছে

Python এ **Variables** এর বৈশিষ্ট্য

আলাদা করে ডেটা টাইপ ঘোষণা করতে হয় না (Dynamic Typing)।

```
x = 10
x = "Hello"
```

একই ভ্যারিয়েবল নতুন ভ্যালু নিয়ে ডেটা টাইপও পরিবর্তন করতে পারে।

সমান চিহ্ন (=) দিয়ে মান অ্যাসাইন করা হয়।
country = "Bangladesh"
এক লাইনে একাধিক ভ্যারিয়েবলেও মান দেওয়া যায়।
a, b, c = 5, 10, 15

Python এ সাধারণ Data Types

Python এ অনেক ধরনের Data Types আছে, তবে সবচেয়ে বেশি ব্যবহৃত কিছু হলো:

- Numeric Types
- int → পূর্ণসংখ্যা (যেমন 10, -5)
- float → দশমিক সংখ্যা (যেমন 3.14, -0.5)
- Text Type
- str → স্ট্রিং (যেমন "Hello", 'Python')
- Sequence Types
- list → পরিবর্তনযোগ্য লিস্ট (যেমন [1, 2, 3])
- tuple → অপরিবর্তনযোগ্য লিস্ট (যেমন (1, 2, 3))
- Mapping Type
- dict → কী-ভ্যালু পেয়ার (যেমন {"name": "Rafi", "age": 25})
- Set Types
- set → অনন্য মানের কালেকশন (যেমন {1, 2, 3})
- Boolean Type
- bool → True বা False

Variables Naming Convention

Python এ ভ্যারিয়েবল নামকরণ করার কিছু নিয়ম ও ভালো অভ্যাস (convention) আছে:

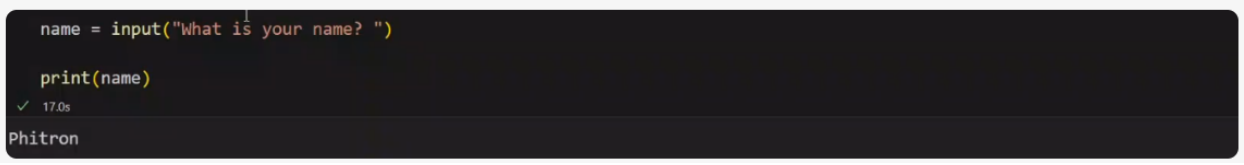
নিয়ম:

- নামের শুরুতে অক্ষর বা আন্ডারস্কোর (_) হতে হবে
- সংখ্যা দিয়ে শুরু করা যাবে না
    - name = "Phitron" # valid
    - _age = 20 # valid
    - 2nd_number = 50 # invalid
- শুধুমাত্র অক্ষর, সংখ্যা ও আন্ডারস্কোর ব্যবহার করা যায়।
- স্পেস বা স্পেশাল ক্যারেক্টার ব্যবহার করা যাবে না।
- ভ্যারিয়েবল case-sensitive।
- age, Age, AGE — তিনটি আলাদা ভ্যারিয়েবল

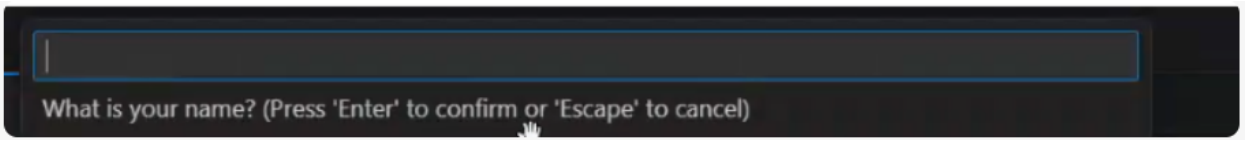
Taking Input in Python

এই মডিউলে আমরা দেখব কিভাবে টার্মিনাল থেকে ডেটা ইনপুট নেয় যায়

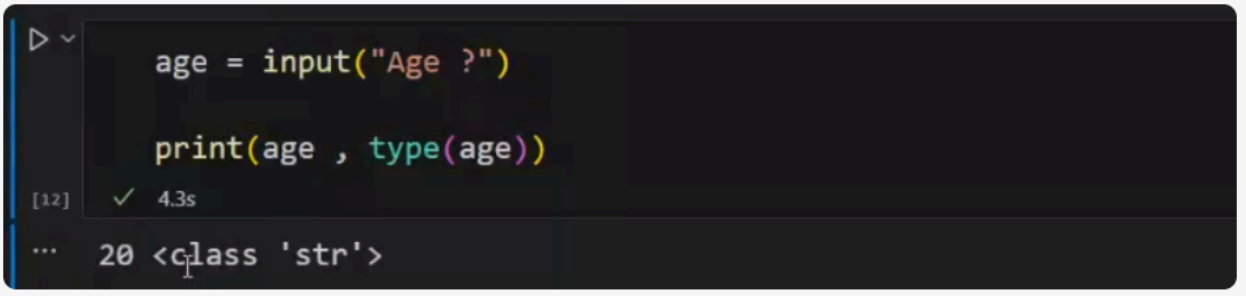
এর জন্য আমরা পাইথনের বিল্ট-ইন input মেথড ইউস করতে পারি



কোড স্নিপেট টি রান করলে VS Code এ ইনপুট নেয়ার জন্য একটি ডায়লগ ওপেন হবে

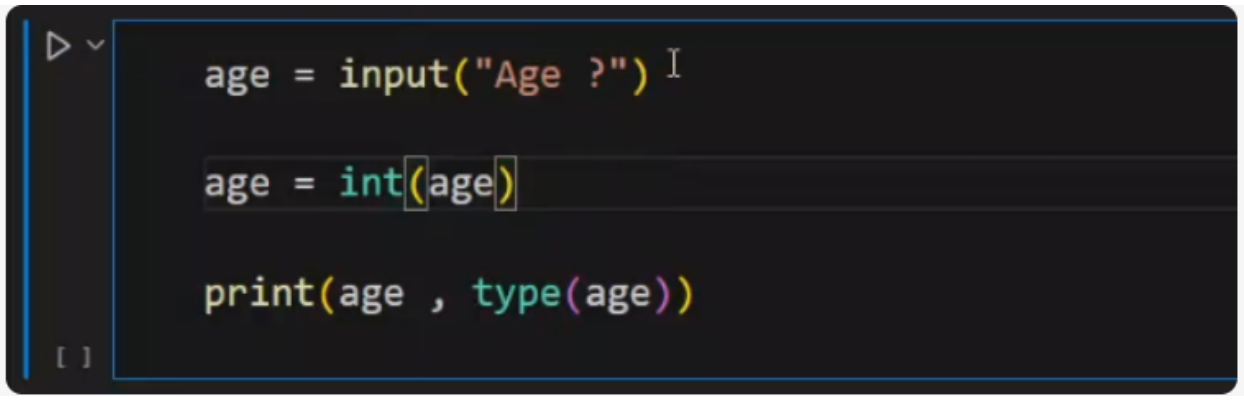


আরো একটি উদাহরন দেখা যাক -



এখানে, আমরা ইনপুট হিসেবে একটি int , 20 দিয়েছি । কিন্তু age এর টাইপ 'str' দেখাচ্ছে ।
এ থেকে বোঝা গেল, ইনপুট নেয়া যে কোন ডেটার টাইপ কিন্তু str - ই হবে।

যদিও প্রয়োজন অনুসারে ডেটার টাইপ পরিবর্তন করে নিতে পারব আমরা -



এখানে, age এর ডেটা টাইপ int- এ cast (রুপান্তর) করা হয়েছে

Operators in Python

Python প্রোগ্রামিং ভাষায় অপারেটর হলো এমন কিছু প্রতীক বা চিহ্ন যেগুলো ভ্যারিয়েবল বা মানের উপর নির্দিষ্ট কাজ সম্পাদন করে। উদাহরণ: +, -, *, / ইত্যাদি।

Arithmetic Operators

এগুলো গাণিতিক হিসাব-নিকাশ করার জন্য ব্যবহার হয়।

উদাহরণ:

x = 10, y = 3

x + y → যোগফল (13)

x - y → বিয়োগফল (7)

x * y → গুণফল (30)

x / y → ভাগফল (3.333...)

x % y → modulus (1)

x // y → পূর্ণসংখ্যা ভাগফল (3)

x ** y → power ($10^3 = 1000$)

Comparison Operators

দুটি মানের মধ্যে তুলনা করার জন্য ব্যবহৃত হয়। ফলাফল সবসময় True অথবা False হয়।

উদাহরণ:

x == y → সমান কি না

x != y → সমান নয় কি না

x > y → বড় কি না

x < y → ছোট কি না

x >= y → বড় বা সমান কি না

x <= y → ছোট বা সমান কি না

Assignment Operators

ভ্যারিয়েবলে মান সেট বা আপডেট করার জন্য ব্যবহার হয়।

উদাহরণ:

x = 5 → মান সেট করা

x += 3 → x = x + 3

x -= 2 → x = x - 2

x *= 4 → x = x * 4

x /= 2 → x = x / 2

x %= 3 → x = x % 3

x **= 2 → x = x ** 2

Membership Operators

একটি মান কোনো সিকোয়েন্স (যেমন লিস্ট, টিউপল, স্ট্রিং) এর মধ্যে আছে কি না তা যাচাই করতে ব্যবহৃত হয়।

উদাহরণ:

"a" in "apple" → True

"b" not in "apple" → True

Identity Operators

দুটি ভ্যারিয়েবল একই অবজেক্টকে নির্দেশ করছে কি না তা যাচাই করতে ব্যবহৃত হয়।

উদাহরণ:

x is y

x is not y

Logical Operator and Precedence, Associativity

Logical Operators কী?

Logical operators হলো এমন অপারেটর যা boolean মানের উপর কাজ করে এবং ফলাফল হিসেবে True বা False প্রদান করে।

Python-এ তিনটি logical operators আছে:

- and
- or
- not

And Operator

দুটি বা তার বেশি boolean মানকে যুক্ত করে।

ফলাফল True হবে শুধুমাত্র যদি সব মান True হয়।

উদাহরণ:

True and True → True

True and False → False

False and False → False

or Operator

দুটি বা তার বেশি boolean মানের মধ্যে যেকোনো একটি True হলে ফলাফল True হয়।

উদাহরণ:

True or False → True

False or False → False

True or True → True

not Operator

একটি boolean মানকে উল্টো করে। True হলে False হয়, False হলে True হয়।

উদাহরণ:

not True → False

not False → True

Operator Precedence Table (Highest to Lowest)

- () - parentheses (grouping)
- ** - exponentiation
- +x, -x, ~x - unary plus, minus, bitwise NOT
- *, /, //, % - multiplication, division, floor division, modulus
- +, - - addition, subtraction
- <<, >> - bitwise shifts
- & - bitwise AND
- ^ - bitwise XOR
- | - bitwise OR
- Comparison: ==, !=, >, <, >=, <=
- not - logical NOT
- and - logical AND
- or - logical OR
- Assignment: =, +=, -=, *=, /=, //=, %=, **=

Problem Solving - Digit Summation

আমরা পাইথন ইউস করে codeforces থেকেীকটি প্রব্লেম সলভ করব

শুরুতেই প্রবলেম টি ভালমত পড়ে নিই-

https://codeforces.com/group/MWSDmqGsZm/contest/219158/problem/F

এবার ধাপে ধাপে প্রবলেম টি সলভ করি -

ইনপুট নেওয়া

Python এ বড় সংখ্যা নেওয়া সম্ভব, তাই 10^18 পর্যন্ত সংখ্যা সমস্যা ছাড়াই নেওয়া যায়।

N, M = map(int, input().split())
এখানে , split() মেথডের সাহায্যে ইনপুট গুলো কে স্পেস দিয়ে আলাদা করা হয়েছে এবং map() এর সাহায্যে সবগুলো ইনপুট কে int এ কনভার্ট করা হয়েছে

last digit বের করা

সংখ্যার শেষ অঙ্ক বের করার জন্য modulus % 10 ব্যবহার করা হয়।

last_digit_N = N % 10
last_digit_M = M % 10
লাস্ট ডিজিট দুইটির যোগফল গণনা

যোগফল:
result = last_digit_N + last_digit_M
ফলাফল প্রিন্ট করা

Python এর print() ব্যবহার করে ফলাফল দেখানো যায়।

কোড

```
N, M = map(int, input().split())

last_digit_N = N % 10
last_digit_M = M % 10

result = last_digit_N + last_digit_M

print(result)
```

Control Flow In Python

Conditional statement in Python

Introduction

Conditional statements প্রোগ্রামকে decision নিতে সাহায্য করে। মানে, নির্দিষ্ট condition true/false এর উপর ভিত্তি করে কোডের আলাদা অংশ execute হবে।

- if → condition true হলে execute হয়
- else → if false হলে fallback হিসেবে execute হয়
- elif → একাধিক শর্ত (multiple conditions) চেক করার জন্য

1. Basic if-else

Example:

```
raining = True

if raining == True:
    print("Rain coat is necessary")
    print("Caution")
else:
    print("Go without a rain coat")
    print("This is a sunny day")
```

Output (যখন raining = True):

```
Rain coat is necessary
Caution
```

Output (যখন raining = False):

```
Go without a rain coat
This is a sunny day
```

এখানে if block শুধুমাত্র তখন চলবে যখন condition true। অন্যথায় else block চলবে।

2. Nested if-else

একটি if-else block এর ভিতরে আরেকটি if-else লেখা যায়।

Example:

```
raining = False
Muddy = True

if raining == True:
    print("Rain coat is necessary")
    if Muddy == True:
        print("Sandal is necessary")
```

```
    else:
        print("Bring Sneakers")
else:
    print("Go without a rain coat")
    print("This is a sunny day")
```

Output:

```
Go without a rain coat
This is a sunny day
```

Nested if ব্যবহার হয় যখন multiple dependent conditions চেক করতে হয়।

3. if-elif-else

একাধিক condition থাকলে elif ব্যবহার করা হয়।

Example:

```
taka = 200

## 500 er beshi hole ghurte jabo
## 400 er beshi hole bashai order korbo
## 300 er beshi hole food court e jabo
## er cheye kom thakle savings

if taka > 500:
    print("ghurte jabo")
elif taka > 400:
    print("Bashai order korbo")
elif taka > 300:
    print("Food court e jabo")
else:
    print("Taka ta save korbo")
```

Output (taka = 200):

```
Taka ta save korbo
```

if-elif-else structure top-to-bottom চেক করে। প্রথম যে condition true হবে সেটি চলবে এবং বাকি condition চেক হবে না।

4. Multiple Conditions with and / or

Example:

```
taka = 1000
raining = True

## 500 er beshi and jodi brishti na hoi
if taka > 500 and raining == False:
    print("ghurte jabo")
```

else:
    print("Taka ta save korbo")
Output (যখন raining = True):
Taka ta save korbo
Output (যখন raining = False):
ghurte jabo
এখানে দুইটা condition একসাথে চেক করা হচ্ছে। and → উভয়ই true হতে হবে, or → যেকোনও একটি true হলেই হবে।

5. Key Takeaways

if একা ব্যবহার করা যায়, কিন্তু সাধারণত if-else pair বেশি ব্যবহৃত হয়।
একাধিক শর্ত থাকলে elif দিয়ে handle করতে হয়।
Logical operator (and, or, not) দিয়ে multiple condition চেক করা যায়।
Nested if readability কমায়, তাই বেশি ব্যবহার না করাই ভালো।

For Loop in Python

Introduction

Programming-এ loop ব্যবহার করা হয় যখন কোনো কাজ বারবার করতে হয়। Python-এ দুটি প্রধান loop রয়েছে:

- for loop → একটি sequence বা iterable এর উপর iterate করে
- while loop → একটি condition true থাকা পর্যন্ত iterate করে

এখানে আমরা for loop নিয়ে আলোচনা করব।

1. Basic Syntax

For Loop Syntax
for variable in iterable:
    # repeat হওয়া কোড
variable → প্রতিবার iterable থেকে একটি element পায়
iterable → list, tuple, string, range ইত্যাদি হতে পারে
Range Syntax
range(stop)   # 0 থেকে stop-1 পর্যন্ত
range(start, stop)   # start থেকে stop-1 পর্যন্ত
range(start, stop, step) # step করে বৃদ্ধি/হ্রাস
List Syntax
list_name = [item1, item2, item3]
List হলো ordered collection

Loop দিয়ে list এর প্রতিটি item অ্যাক্সেস করা যায়

2. Basic for Loop with range

range(n) একটি সংখ্যা sequence তৈরি করে 0 থেকে n-1 পর্যন্ত।

Example:

```
for i in range(10):
    print(i, "Hello World")
```

Output:

```
0 Hello World
1 Hello World
2 Hello World
3 Hello World
4 Hello World
5 Hello World
6 Hello World
7 Hello World
8 Hello World
9 Hello World
```

এখানে loop 10 বার ঘুরেছে (0 থেকে 9 পর্যন্ত)।

3. range এবং list

range কে list এ রূপান্তর করলে পুরো sequence দেখা যায়।

Example:

```
for i in range(10):
    print(i, "Hello World")

print(list(range(10)))
```

Output:

```
0 Hello World
1 Hello World
...
9 Hello World
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

4. Custom Start এবং Step Value

range(start, stop, step) দিয়ে শুরু মান, শেষ মান, এবং ধাপ নির্দিষ্ট করা যায়।

Example:

```
for i in range(1, 11, 2):
    print(i, "Hello World")
```

Output:

1 Hello World
3 Hello World
5 Hello World
7 Hello World
9 Hello World
এখানে loop 1 থেকে শুরু হয়েছে এবং প্রতিবার 2 করে বৃদ্ধি পাচ্ছে।

5. Nested for Loop
একটি for loop এর ভিতরে আরেকটি for loop ব্যবহার করলে nested loop তৈরি হয়।

Example:
```
for i in range(1, 4):
    for j in range(1, 5):
        print(i, j)
```
Output:
1 1
1 2
1 3
1 4
2 1
2 2
2 3
2 4
3 1
3 2
3 3
3 4
Nested loop সাধারণত table তৈরি, pattern print, combinations বের করার জন্য ব্যবহৃত হয়।

6. Key Takeaways
for loop iterable (যেমন list, tuple, string, dictionary, range) এর উপর কাজ করে।
range(n) → 0 থেকে n-1 পর্যন্ত সংখ্যা তৈরি করে।
range(start, stop, step) ব্যবহার করে custom sequence তৈরি করা যায়।
list ব্যবহার করে একাধিক data collection loop করা যায়।
Nested for loop দিয়ে multiple dimension এ কাজ করা সম্ভব।

While Loop in Python

Introduction
while loop ব্যবহার করা হয় যখন কোনো condition true থাকা পর্যন্ত কোড বারবার চালাতে হবে। এটি মূলত condition-driven iteration। যতক্ষণ পর্যন্ত condition সত্য থাকে, ততক্ষণ loop চলবে।

1. Basic Syntax
While Loop Syntax
```
while condition:
    # repeat হওয়া কোড
```
condition → একটি boolean expression। condition true হলে loop চালু থাকবে।
Condition false হলে loop থেমে যাবে।

2. Basic Example
Example:
```
count = 1

while count < 10:
    print(count, "hello world")
    count = count + 1

print(count)
```
Output:
1 hello world
2 hello world
3 hello world
4 hello world
5 hello world
6 hello world
7 hello world
8 hello world
9 hello world
10
এখানে loop চলেছে যতক্ষণ পর্যন্ত count < 10 শর্ত true ছিল। শেষবার loop থেমে গেলে count এর মান 10 হয়েছে।

3. Key Points
Initialization: সাধারণত loop শুরু করার আগে counter variable initialize করতে হয়।
Update: প্রতিবার iteration শেষে counter update করতে হয়, না হলে infinite loop হবে।
Exit Condition: condition false হলে loop বন্ধ হবে।

4. Common Mistake
যদি update না করা হয়, তাহলে infinite loop তৈরি হবে:

```
count = 1
while count < 10:
    print(count)
    # এখানে count update করা হয়নি, তাই loop কখনো থামবে না
```

5. Comparison with For Loop

for loop → যখন নির্দিষ্ট সংখ্যক বার repeat করতে হয়।
while loop → যখন repeat করার সংখ্যা আগে থেকে নির্দিষ্ট না, condition এর উপর নির্ভর করে।

6. Key Takeaways

while loop condition true থাকা পর্যন্ত চলে।
Counter initialize ও update করা অত্যন্ত জরুরি।
Infinite loop এড়াতে exit condition নিশ্চিত করতে হবে।

Break and Continue Statements in Python

Introduction

Loop এর execution flow নিয়ন্ত্রণ করার জন্য Python এ break এবং continue statement ব্যবহার করা হয়। এগুলো loop-এর normal flow পরিবর্তন করে।

break → সম্পূর্ণ loop বন্ধ করে দেয় এবং control loop-এর বাইরে চলে যায়।
continue → current iteration বাদ দিয়ে loop-এর পরবর্তী iteration শুরু করে।

1. Syntax

Break Syntax

```
for variable in iterable:
    if condition:
        break
    # বাকি কোড
```

অথবা

```
while condition:
    if condition2:
        break
    # বাকি কোড
```

Continue Syntax

```
for variable in iterable:
    if condition:
        continue
    # বাকি কোড
```

অথবা

```
while condition:
    if condition2:
        continue
    # বাকি কোড
```

2. Example of Continue

Example:

```
sum = 0

for i in range(1, 11):
    print(i, "is processing")
    if i % 2 == 0:  # even number skip হবে
        continue
    sum += i
    print(i, "is added to the sum")
```

Output:

```
1 is processing
1 is added to the sum
2 is processing
3 is processing
3 is added to the sum
4 is processing
5 is processing
5 is added to the sum
...
9 is processing
9 is added to the sum
10 is processing
```

এখানে even সংখ্যা skip হয়েছে কারণ continue ব্যবহার করা হয়েছে।

3. Example of Break

Example 1: For Loop

```
accuracy = 30

for i in range(100):
    accuracy += 1
    print(accuracy)
    if accuracy == 100:
        break
```

Output (সংক্ষেপে):

```
31
32
```

...
100
Loop 100-এ পৌঁছানোর সাথে সাথেই break হয়েছে এবং থেমে গেছে।

Example 2: While Loop
```
count = 1

while True:
    print(count)
    if count == 5:
        break
    count += 1
```
Output:
1
2
3
4
5
এখানে while True infinite loop তৈরি করেছে, কিন্তু break দিয়ে 5 এ পৌঁছে loop বন্ধ করা হয়েছে।

4. Key Takeaways
break → loop সম্পূর্ণভাবে থামিয়ে দেয়।
continue → শুধু current iteration বাদ দেয়, loop চলতে থাকে।
break এবং continue উভয়ই for এবং while loop-এ কাজ করে।
এগুলো ব্যবহার করার সময় readability মাথায় রাখা উচিত।

Problem 1 - k: Min and Max

Theory Discussion
কোনো তিনটি সংখ্যার মধ্যে minimum (সবচেয়ে ছোট সংখ্যা) এবং maximum (সবচেয়ে বড় সংখ্যা) বের করার জন্য আমরা if statement ব্যবহার করতে পারি। Python-এ তুলনা করার জন্য <, >, <=, >= অপারেটর ব্যবহার করা হয়।

Steps:

প্রথমে একটি সংখ্যা min এবং max হিসেবে ধরা হয়।
এরপর অন্য সংখ্যা গুলোকে if condition ব্যবহার করে min এবং max এর সাথে তুলনা করা হয়।
যদি কোনো সংখ্যা min এর চেয়ে ছোট হয় তাহলে সেটাই নতুন min হবে।
আবার যদি কোনো সংখ্যা max এর চেয়ে বড় হয় তাহলে সেটাই নতুন max হবে।

Basic Syntax (if statement দিয়ে তুলনা)

```
if y < min:
   min = y
if z < min:
   min = z

if y > max:
   max = y
if z > max:
   max = z
```

Example Code

```
# তিনটি সংখ্যা input হিসেবে নেয়া
numbers = input().split()

x = int(numbers[0])
y = int(numbers[1])
z = int(numbers[2])

min = x
max = x

# minimum বের করা
if y < min:
   min = y
if z < min:
   min = z

# maximum বের করা
if y > max:
   max = y
if z > max:
   max = z

print(min, max)
```

Example Run (Output)
Input:

```
1 2 3
```

Output:

```
1 3
```

Input:

-1 -2 -3
Output:

-3 -1
Input:

10 20 -5
Output:

-5 20

Problem Link:
https://codeforces.com/group/MWSDmqGsZm/contest/219158/problem/K

Problem 2 - C - Even,Odd,Positive and Negative

Theory Discussion

এই সমস্যায় আমাদের কাজ হলো একটি সংখ্যার তালিকা থেকে চারটি জিনিস গণনা করা:

Even numbers (যে সংখ্যা 2 দিয়ে বিভাজ্য)
Odd numbers (যে সংখ্যা 2 দিয়ে বিভাজ্য নয়)
Positive numbers (যে সংখ্যা 0 এর চেয়ে বড়)
Negative numbers (যে সংখ্যা 0 এর চেয়ে ছোট)
Python-এ কোনো সংখ্যা even বা odd চেক করতে % (modulus) অপারেটর ব্যবহার করা হয়। উদাহরণ:

```
if num % 2 == 0:
    # Even number
else:
    # Odd number
```

একইভাবে positive বা negative চেক করা যায়:

```
if num > 0:
    # Positive
elif num < 0:
    # Negative
```

Basic Syntax

```
for num in numbers:
    if num % 2 == 0:
```

```
        even += 1
    else:
        odd += 1

    if num > 0:
        positive += 1
    elif num < 0:
        negative += 1
```

Example Code

```
# Input নেয়া
n = int(input())
numbers = list(map(int, input().split()))

# কাউন্টার initialize করা
even = 0
odd = 0
positive = 0
negative = 0

for num in numbers:
    if num % 2 == 0:
        even += 1
    else:
        odd += 1

    if num > 0:
        positive += 1
    elif num < 0:
        negative += 1

print("Even:", even)
print("Odd:", odd)
print("Positive:", positive)
print("Negative:", negative)
```

Example Run (Output)
Input:

```
5
-5 0 -3 -4 12
```

Output:

Even: 3
Odd: 2
Positive: 1
Negative: 3

Problem Link:
https://codeforces.com/group/MWSDmqGsZm/contest/219432/problem/C

Problem 3 -Digits

Theory Discussion

এই সমস্যায় আমাদের একটি সংখ্যা N কে ডান দিক থেকে বাম দিকে প্রতিটি digit আলাদা করে space দিয়ে প্রিন্ট করতে হবে। একাধিক test case দেওয়া থাকবে।

Python-এ সংখ্যার digit গুলো বের করার দুটি উপায় আছে:

String conversion → সংখ্যাকে string-এ রূপান্তর করে index ব্যবহার করা।
Mathematical method → সংখ্যা থেকে বারবার % 10 এবং // 10 ব্যবহার করে digit বের করা।

Basic Syntax (Mathematical approach)

```
while n > 0:
    digit = n % 10
    print(digit, end=" ")
    n //= 10
```

Example Code

```
# Input নেয়া
T = int(input())
for _ in range(T):
    n = int(input())

    digits = []
    if n == 0:
        digits.append(0)
    else:
        while n > 0:
            digits.append(n % 10)
            n //= 10

    print(" ".join(map(str, digits)))
```

Example Run (Output)
Input:

4
121
39
123456
1200# 2.7 Problem 3 - Digits

Theory Discussion

এই সমস্যায় আমাদের একটি সংখ্যা **N** কে ডান দিক থেকে বাম দিকে প্রতিটি digit আলাদা করে space দিয়ে প্রিন্ট করতে হবে। একাধিক **test case** দেওয়া থাকবে।

Python-এ সংখ্যার digit গুলো বের করার দুটি উপায় আছে:

1. **String conversion** → সংখ্যাকে string-এ রূপান্তর করে index ব্যবহার করা।
2. **Mathematical method** → সংখ্যা থেকে বারবার `% 10` এবং `// 10` ব্যবহার করে digit বের করা।

Basic Syntax (Mathematical approach)

```python
while n > 0:
    digit = n % 10
    print(digit, end=" ")
    n //= 10
```

Example Code

```python
# Input নেয়া
T = int(input())
for _ in range(T):
    n = int(input())

    digits = []
    if n == 0:
        digits.append(0)
    else:
```

```
        while n > 0:
            digits.append(n % 10)
            n //= 10

    print(" ".join(map(str, digits)))
```

Example Run (Output)

Input:

```
4
121
39
123456
1200
```

Output:

```
1 2 1
9 3
6 5 4 3 2 1
0 0 2 1
```

Output:

1 2 1
9 3
6 5 4 3 2 1
0 0 2 1

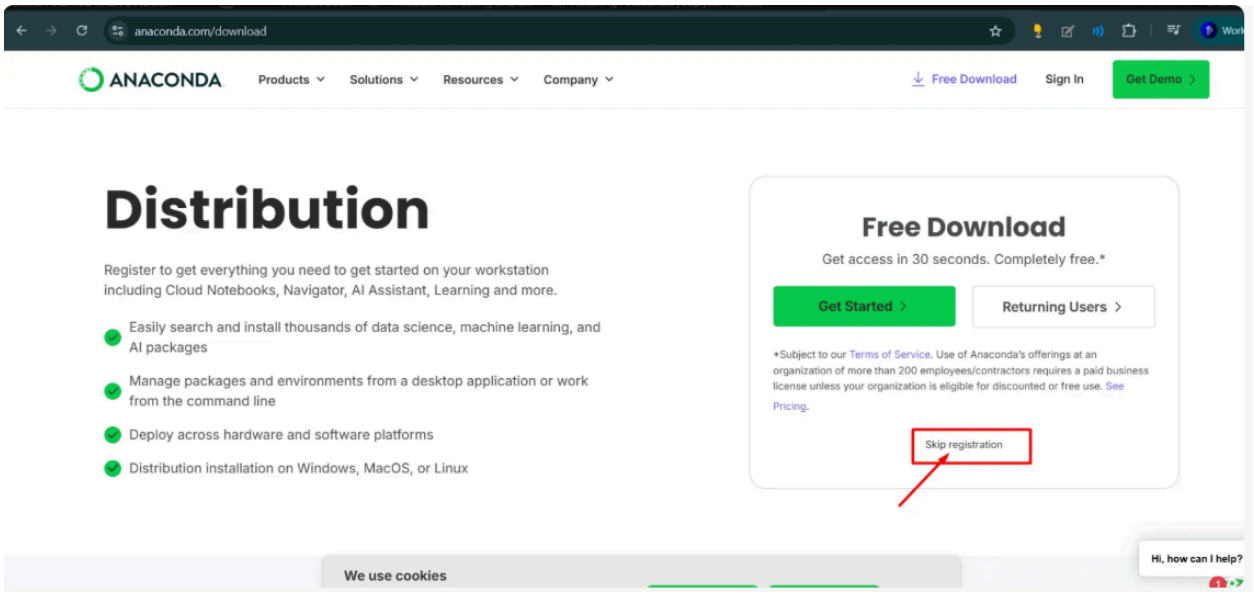
Problem Link:
https://codeforces.com/group/MWSDmqGsZm/contest/219432/problem/Q

Installing Anaconda , Jupyter Notebook , Google Collab

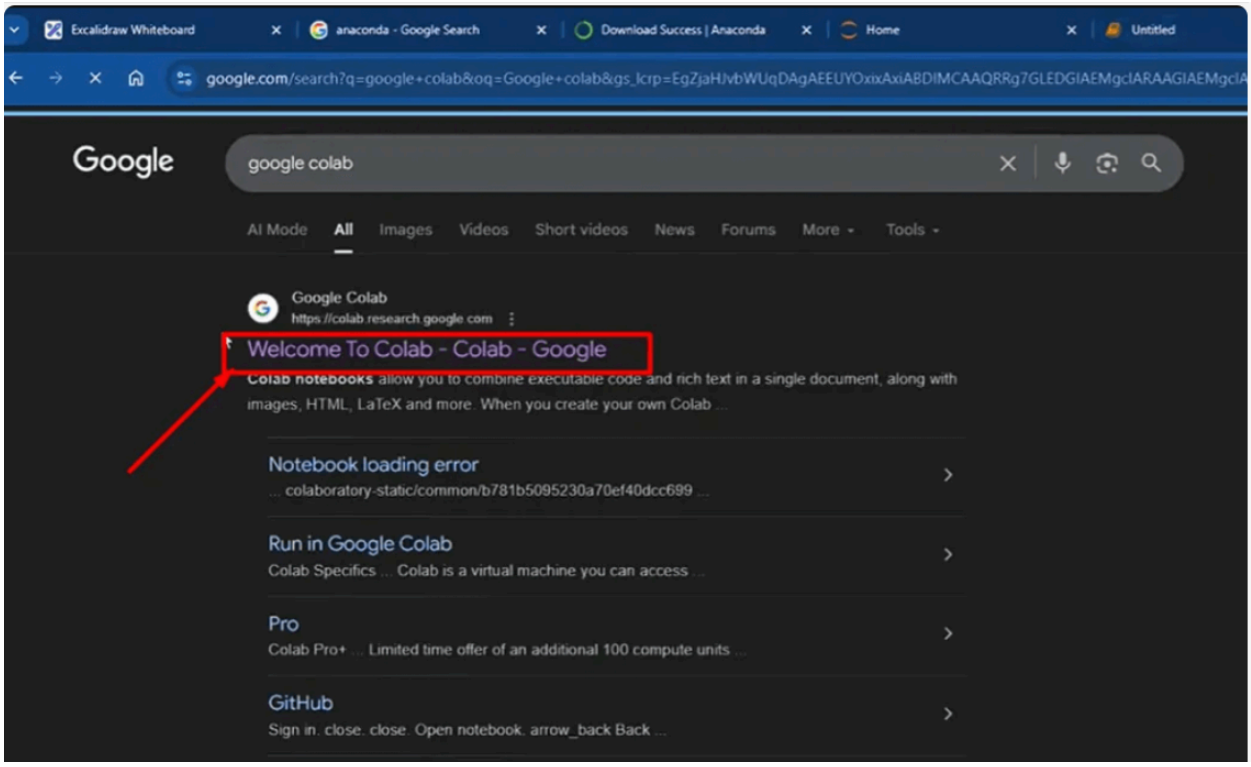
Anaconda Installation

Anaconda install করতে এই https://www.anaconda.com/download লিংকে প্রবেশ করুন।

এরপর Skip registration এ ক্লিক করুন।

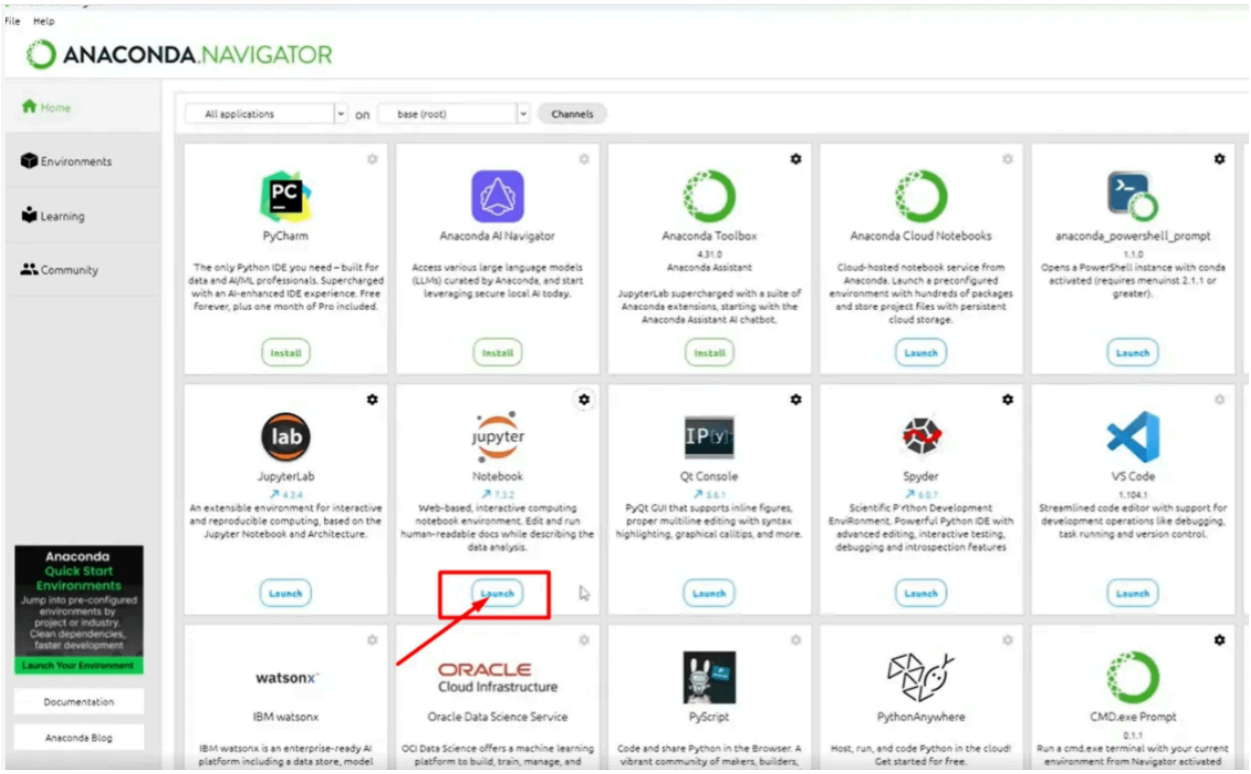


এরপর download page এ নিয়ে যাবে। সেইখানে Distribution Installers টি আপনার Operating system অনুযায়ী download করে নিবেন।



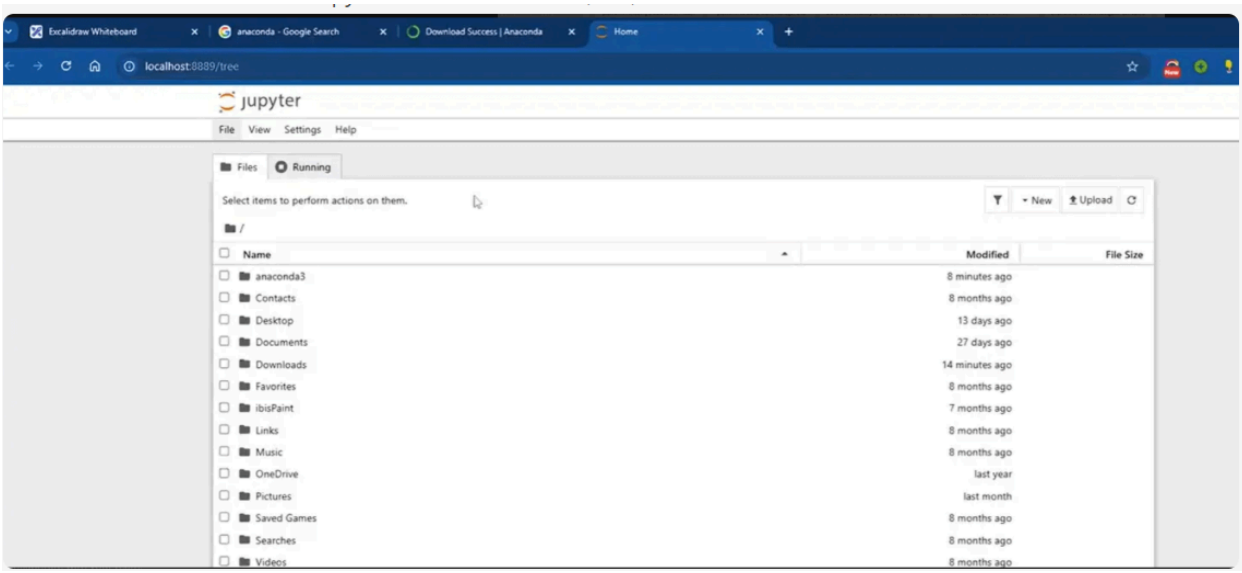
এরপর installer টি দিয়ে Anaconda টি install করে নিবেন।

Install হয়ে গেলে Anaconda Navigator Software টি ওপেন করবেন। এরপর সেইখানে Jupyter Notebook টি launch করবেন।

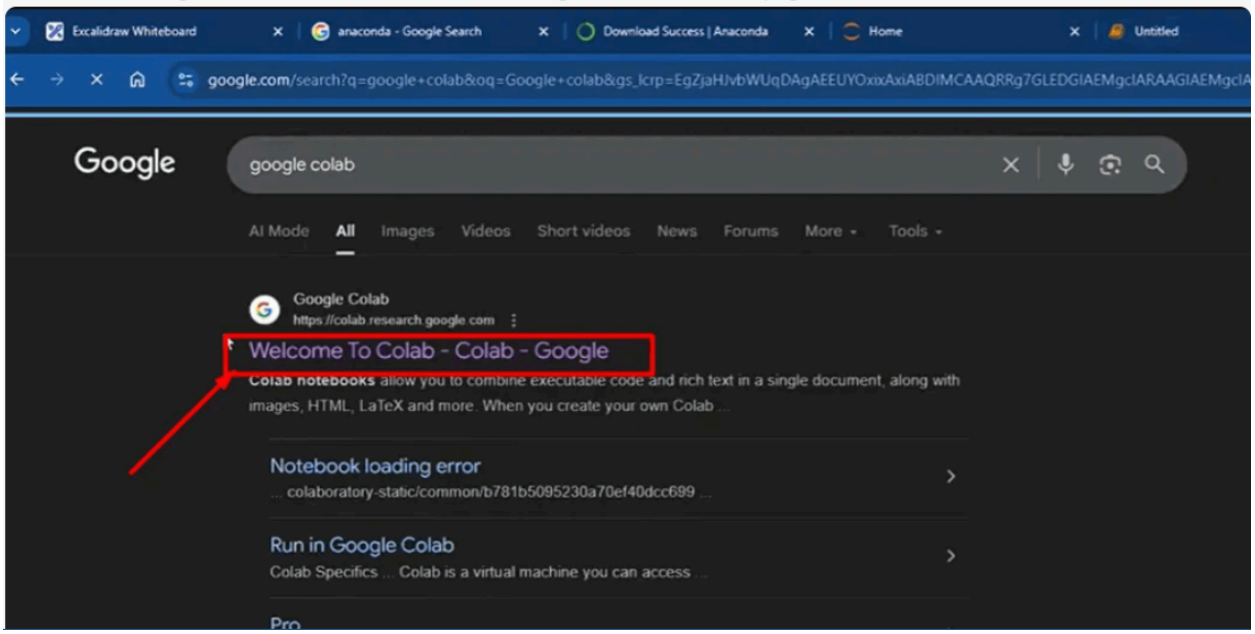


Jupyter Notebook

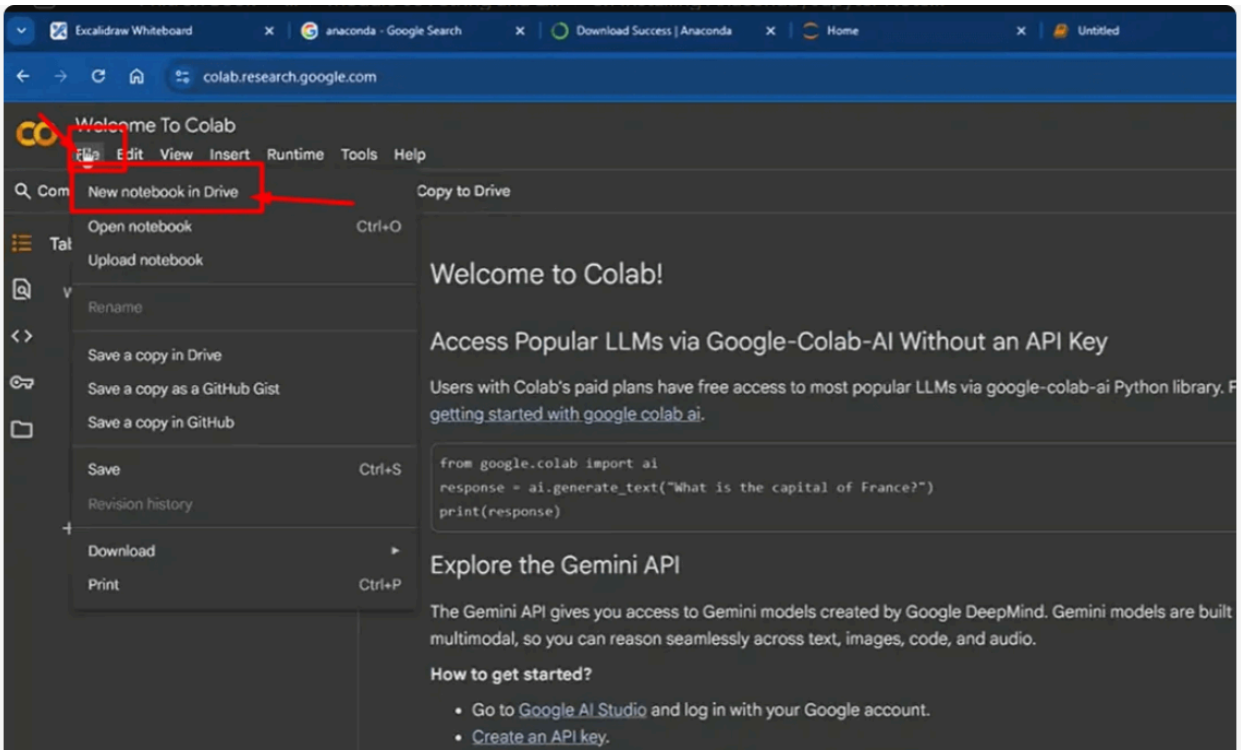
Launch করা পর দেখবে Browser এ Jupyter Notebook টি ওপেন হয়ে গিয়েছে।



Google Collab



Browser গিয়ে Google Collab লিখে search দিবেন, এরপর Google Collab এর official page টি চলে যাবেন।



এরপর আপনি file এ যাবে সেইখানে New notebook in Drive এ ক্লিক করুন।

String Indexing and Slicing

1. String কী?

Python এ string হলো character গুলোর একটি সিকোয়েন্স। যেকোনো text ' ' [single quote] অথবা " " [double quote] দিয়ে ঘিরে দিলে সেটা string হয়ে যায়। একাধিক লাইনের লেখার জন্য """ """ [triple quotes] ব্যবহার করা হয়।

✅ Example: Simple String

pythonprompt = "what is your name?"
print(prompt)
print(type(prompt))
Output:

textwhat is your name?
<class 'str'>
এখানে prompt হলো string , যা <class 'str'> টাইপের।

✅ Example: Multiline String

pythonmessage = """tell me about yourself ?
what are the issues you've been facing while learning to code ?
How are you tackling them ?"""
print(message)
Output:

texttell me about yourself ?
what are the issues you've been facing while learning to code ?
How are you tackling them ?
তিন লাইন একসাথে প্রিন্ট হয়েছে কারণ triple quotes ব্যবহার করা হয়েছে।

2.Indexing

String এর প্রতিটি অক্ষরের একটি ইনডেক্স নম্বর থাকে, যা ০ থেকে শুরু হয়।

Example: "hello world"

h e l l o w o r l d
0 1 2 3 4 5 6 7 8 9 10
✅ Example: Index দিয়ে Character বের করা

pythonstring = "hello world"

print(string[7])
Output:

texto
string হলো ৭ নম্বর index এর Character, অর্থাৎ 'o'।

3. Slicing

String এর নির্দিষ্ট অংশ পেতে আমরা Slicingব্যবহার করি।

Syntax:

textstring[start:end]
start থেকে শুরু করে end-1 পর্যন্ত অংশ নেয়।

✅ Example: Simple Slicing

pythonfirst_message = message[0:24]
print(first_message)
Output:

texttell me about yourself ?
message[0:24] মানে প্রথম ২৪টি অক্ষর (০-২৩) নেয়া হয়েছে।

✅ Example: Negetive index ব্যবহার

pythonprint(first_message[-1])
print(first_message[-2])
print(first_message[-3])
print(first_message[-4])
print(first_message[-5])
Output:

text?
f
l
e
Negetive index মানে string এর শেষ থেকে গনণা শুরু, -1 হলো শেষ character।

✅ Example: String Reverse

pythonnew_first_message = first_message[::-1]

print(new_first_message)
Ouput:

text? flesruoy tuoba em llet
[::-1] দিলে srting reverse হয়ে যায়।

String Methods

Python String Methods

Python এ string নিয়ে কাজ করার জন্য অনেক বিল্ট-ইন method আছে।

এগুলো string কে lowercase অক্ষরে রূপান্তর, substring খুঁজে বের করা, replace করা ইত্যাদি কাজে ব্যবহার হয়।

1. lower() → lowercase এ রূপান্তর

string = "Welcome to Phitron ML Course , Phitron ML , Hello ML"
processed_string = string.lower()
print(processed_string)
Output:

welcome to phitron ml course , phitron ml , hello ml

lower() string এর সব character কে lowercase এ রূপান্তর করে।

2. len() → String এর Length বের করা

print(len(string))
Output:

52

len() string এ মোট কতগুলো character আছে সেটা বলে।

3. in Operator → Substring আছে কি না চেক করা

print("phitron" in processed_string)

Output:

True

এখানে "phitron" শব্দটি processed_string এর মধ্যে আছে, তাই True এসেছে।

4. find() → প্রথমবার substring কোথায় পাওয়া গেছে সেইটি বের করা

```
shuru_index = processed_string.find("phitron")
print(shuru_index)
```

Output:

11

"phitron" প্রথমবার শুরু হয়েছে index 11 থেকে।

5. rfind() → সর্বশেষবার substring কোথায় পাওয়া গেছে সেইটি বের করা

```
shesh_index = processed_string.rfind("phitron")
print(shesh_index)
```

Output:

32

"phitron" সর্বশেষবার পাওয়া গেছে index 32 থেকে।

6. count() → Substring কয়বার আছে

```
count = processed_string.count("phitron")
print(count)
```

Output:

2

"phitron" মোট ২ বার এসেছে string এর মধ্যে।

7. replace() → Substring Replace করা

```
processed_string = string.replace("ML", "AI/ML")
print(processed_string)
```

Output:

```
Welcome to Phitron AI/ML Course , Phitron AI/ML , Hello AI/ML
```

এখানে "ML" শব্দটিকে "AI/ML" দিয়ে replace করা হয়েছে।

Summary

- lower() → সব অক্ষর ছোট হাতের করে।
- len() → string এর দৈর্ঘ্য বের করে।
- in → substring আছে কি না চেক করে।
- find() → substring প্রথম কোথায় পাওয়া গেছে বলে।
- rfind() → substring শেষবার কোথায় পাওয়া গেছে বলে।
- count() → substring কয়বার আছে সেটা বলে।
- replace() → substring কে নতুন word দিয়ে পরিবর্তন করে।

আরও বিস্তারিত জানতে এইখানে যেতে পারো https://www.w3schools.com/python/python_ref_string.asp

String Splitting, Joining & Formatted String

Python Splitting, Joining & Formatted String

1. split() → String কে List এ ভাগ করা

```
prompt = "what is Phitron ?"
tokens = prompt.split()
print(tokens)
print(type(tokens))
```

Output:

```
['what', 'is', 'Phitron', '?']
<class 'list'>
```

split() string কে শব্দ অনুযায়ী কেটে list বানায়। ডিফল্টভাবে space অনুযায়ী ভাগ হয়।\

2. join() → List থেকে String বানানো

```
sentence = "-".join(tokens)
print(sentence)
print(type(sentence))
```

Output:

```
what-is-Phitron-?
<class 'str'>
```

এখানে "-".join(tokens) দিয়ে list এর প্রতিটি শব্দকে - দিয়ে জোড়া লাগানো হয়েছে।

3. Formatted String (f-string)

Formatted string ব্যবহার করে string এর মধ্যে ভ্যারিয়েবল embed করা যায়।

```
name = "Adil"
age = 23
height = 5.1123445

info = f"His name is {name.upper()}. He is {age} years old. He is {height:.3} feet tall.\\n"
print(info)
```

Output:

```
His name is ADIL. He is 23 years old. He is 5.11 feet tall.
```

{name.upper()} → name কে uppercase এ প্রিন্ট করেছে। {height:.3} → height কে ৩ ডিজিট পর্যন্ত সীমাবদ্ধ করেছে।

4. Floating Value Format

```
model_accuracy = 0.83333
print(f"the model accuracy is {model_accuracy:.2f}")
```

Output:

```
the model accuracy is 0.83
```

এখানে {model_accuracy:.2f} মানে হলো দশমিকের পর ২ ঘর পর্যন্ত দেখানো।

List Basics and Methods

Python List

1. Declaration

Python এ list হলো একাধিক item একসাথে রাখার জন্য ব্যবহৃত data structure।

list [ ] এর ভিতরে রাখা হয়।

```
numbers = [100,20,30,40,50,100,100]     # int list
float_numbers = [10.20,3.15,2.4,2.7,3.5] # float list
fruits = ['apple', 'orange' , 'lichi']   # string list
mix = [10 , 10.20 , 'apple']             # mixed list
list_er_moddhe_list =[ [100,20,30] , [14,12,31] , [13,12,12] ] # nested list
```

list এর মধ্যে সংখ্যা, ভাসমান সংখ্যা, স্ট্রিং, এমনকি list ও থাকতে পারে।

2. Access

```
print(numbers[2])
numbers[2] = 90
print(numbers[2])
new_list = numbers[0:3]
print(new_list)
```

Output:

```
30
90
[100, 20, 90]
```

numbers[2] → index 2 এর মান (30) বের করে। পরে সেটিকে পরিবর্তন (30 → 90) করা হয়েছে।
numbers[0:3] → slicing করে প্রথম ৩টি element নেয়া হয়েছে।

3. Modification

✅ আইটেম যোগ করা → append()

```
numbers.append(80)
print(numbers)
```

Output:

```
[100, 20, 30, 40, 50, 100, 100, 80]
```

append() সবসময় শেষে একটি আইটেম যোগ করে।

✅ যেকোনো জায়গায় আইটেম ঢোকানো → insert()

```
numbers.insert(0,45)
print(numbers)
```
Output:

```
[45, 100, 20, 30, 40, 50, 100, 100, 80]
```

insert(index, value) ব্যবহার করে নির্দিষ্ট index এ মান ঢোকানো যায়।

✅ শেষের মান remove করা → pop()

```
numbers.pop()
print(numbers)
```
Output:

```
[45, 100, 20, 30, 40, 50, 100, 100]
```

pop() ডিফল্টভাবে list এর শেষের মান remove করে ফেলে।

✅ নির্দিষ্ট মান remove করা → remove()

```
numbers.remove(100)
print(numbers)
```
Output:

```
[45, 20, 30, 40, 50, 100, 100]
```

remove(value) দিলে list থেকে প্রথম occurrence remove করে ফেলে।

4. Sorting

✅ ছোট থেকে বড় → sort()

numbers.sort()
print(numbers)
Output:

[20, 30, 40, 45, 50, 100, 100]

✅ reverse করে দেওয়া → reverse()

numbers.reverse()
print(numbers)
Output:

[100, 100, 50, 45, 40, 30, 20]

✅ বড় থেকে ছোট → sort(reverse=True)

numbers.sort(reverse=True)
print(numbers)
Output:

[100, 100, 50, 45, 40, 30, 20]

reverse=True দিলে list কে descending order এ সাজায়।

Summary

- List → একাধিক মান একসাথে রাখার জন্য ব্যবহৃত হয়।
- Access → index/slicing ব্যবহার করে মান বের করা যায়।
- append() → শেষে মান যোগ করে।
- insert() → যেকোনো index এ মান ঢোকায়।
- pop() → শেষের মান remove করে ফেলে।
- remove() → নির্দিষ্ট মান remove করে ফেলে।
- sort() → ascending order এ সাজায়।
- reverse() → list উল্টে দেয়।
- sort(reverse=True) → descending order এ সাজায়।

List as Stack and Queue

Python List as Stack & Queue

1. List as Stack

Stack হলো LIFO (Last In First Out) ডেটা স্ট্রাকচার।

নতুন element সবসময় উপরে যোগ হয় → append() দিয়ে।

element সবসময় উপর থেকে মুছে ফেলা হয় → pop() দিয়ে।

✅ Example: Stack Implementation

```
stack = []
# stack এ push করা
stack.append(1)
stack.append(2)
stack.append(3)
stack.append(4)
stack.append(5)
print(f"top element : {stack[-1]}")

# top element মুছে ফেলা
stack.pop()

# নতুন top element
print(f"top element : {stack[-1]}")
```

Output:

```
top element : 5
top element : 4
```

stack[-1] → সবসময় top element access করতে ব্যবহার হয়।

pop() → top element সরিয়ে দিতে ব্যবহার হয়।

2. List as Queue

Queue হলো FIFO (First In First Out) ডেটা স্ট্রাকচার।

নতুন element সবসময় শেষে যোগ হয় → append() দিয়ে।

element সবসময় সামনে থেকে reomove করা হয় → pop(0) দিয়ে।

✅ Example: Queue Implementation

```
queue = []
# enqueue (insertion)
queue.append(1)
queue.append(2)
queue.append(3)
queue.append(4)
queue.append(5)

# dequeue (removal)
print(queue.pop(0))

# front element access
print(f"front e {queue[0]}")

queue.pop(0)
print(f"front e {queue[0]}")
```

Output:

```
1
front e 2
front e 3
```

queue.pop(0) → প্রথম element সরিয়ে দেয়।

queue[0] → front element access করার জন্য ব্যবহার হয়।

Summary
Stack (LIFO)

append() → push (top এ যোগ করা)
pop() → pop (top থেকে সরানো)
stack[-1] → top element
Queue (FIFO)

append() → enqueue (rear এ যোগ করা)
pop(0) → dequeue (front থেকে সরানো)
queue[0] → front element

List Comprehension

Python List Comprehension

1. List Comprehension কী?

Python এ list comprehension হলো list বানানোর সংক্ষিপ্ত ও সুন্দর উপায়।

List Comprehension দিয়ে এক লাইনের মধ্যে loop + condition লিখে নতুন list তৈরি করা যায়।

Syntax:

[expression for item in iterable if condition]

2. সাধারণ পদ্ধতি (Naive way)

```
even = []
for i in range(101):
    if i % 2 == 0:
        even.append(i)

print(even)
```

এখানে loop চালিয়ে শর্ত দিয়ে list বানানো হয়েছে।

3. List Comprehension পদ্ধতি

```
random_list = [x+10 for x in range(1,101) if x%2!=0]
print(random_list)
```

Output:

[11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67, 69, 71, 73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 99, 101, 103, 105, 107, 109]

এখানে range(1,101) থেকে সব বিজোড় সংখ্যা নিয়ে এর সাথে +10 যোগ করে list তৈরি করেছে।

4. List Comprehension দিয়ে String Process করা

```
fruits = ['APPLE' , 'ORange' , 'liCHi']
```

```
upper_fruits = [fruit.lower() for fruit in fruits]
print(upper_fruits)
```
Output:

['apple', 'orange', 'lichi']

এখানে প্রতিটি fruit কে .lower() করে ছোট হাতের অক্ষরে রূপান্তর করা করেছে।

Build in Data Structure

Tuples in Python

Tuple কী**?**

Tuple হলো একটি ordered কিন্তু immutable data structure। অর্থাৎ, এটি list-এর মতোই ordered collection হলেও এর মান পরিবর্তন করা যায় না। একবার Tuple তৈরি হলে এর মান, আকার বা গঠন পরিবর্তন করা সম্ভব নয়।

উদাহরণ:

```
t = (1, 2, 3, 4)
print(t)
```

আউটপুট:

(1, 2, 3, 4)

Tuple কিভাবে তৈরি করা হয়
Tuple তৈরি করতে সাধারণত parentheses ( ) ব্যবহার করা হয়।

```
t1 = (10, 20, 30)
print(type(t1))  # Output: <class 'tuple'>
```

একটি একক (single) মানের Tuple তৈরি করার সময় অবশ্যই শেষে কমা (,) দিতে হয়।

```
t2 = (5,)  # একক Tuple
print(type(t2))  # Output: <class 'tuple'>
```

Tuple Indexing ও Accessing

Tuple-এর প্রতিটি মানের একটি index থাকে যা 0 থেকে শুরু হয়।

```
numbers = (10, 20, 30, 40)
print(numbers[0])  # Output: 10
print(numbers[-1]) # Output: 40
```

Tuple থেকে slicing ব্যবহার করেও নির্দিষ্ট অংশ নেওয়া যায়।

```
print(numbers[1:3])  # Output: (20, 30)
```

Tuple Immutable হওয়ার মানে কী

Tuple তৈরি করার পরে এর মান পরিবর্তন করা যায় না। নিচের উদাহরণটি দেখো:

```
t = (1, 2, 3)
t[0] = 10  # এটি error দেবে
```

Error: TypeError: 'tuple' object does not support item assignment

Tuple Concatenation এবং Repetition

Tuple-গুলো একত্রিত করা বা পুনরাবৃত্তি করা যায় + এবং * অপারেটর ব্যবহার করে।

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)   # Output: (1, 2, 3, 4)
print(t1 * 2)    # Output: (1, 2, 1, 2)
```

Tuple Methods

Tuple-এর জন্য কিছু built-in method আছে যেমন:

Method কাজ
count(x) Tuple-এ x কতবার আছে তা গণনা করে
index(x) Tuple-এ x-এর প্রথম অবস্থান (index) দেয়
উদাহরণ:

```
nums = (1, 2, 2, 3)
print(nums.count(2))  # Output: 2
print(nums.index(3))  # Output: 3
```

Tuple এবং List-এর মধ্যে পার্থক্য

বৈশিষ্ট্য	List	Tuple
Mutable	✅	❌
Syntax	[]	()
Performance	তুলনামূলক ধীর	তুলনামূলক দ্রুত
ব্যবহার	পরিবর্তনযোগ্য ডেটা	স্থির বা অপরিবর্তনীয় ডেটা

মূল পয়েন্টসমূহ

- Tuple হলো immutable ordered collection।
- একবার Tuple তৈরি হলে এর মান পরিবর্তন করা যায় না।
- Tuple-এ indexing, slicing, concatenation সম্ভব।
- Tuple সাধারণত এমন ডেটা সংরক্ষণের জন্য ব্যবহৃত হয় যা পরিবর্তন করা প্রয়োজন নেই।

Sets In python

Set কী?

Set হলো একটি unordered এবং mutable data structure, যেখানে duplicate value রাখা যায় না। Set সাধারণত unique elements রাখার জন্য ব্যবহৃত হয়।

উদাহরণ:

```
s = {1, 2, 3, 3, 4}
print(s)
```

আউটপুট:

```
{1, 2, 3, 4}
```

(এখানে duplicate মান 3 একবারই থাকবে।)

Set create করা

Set তৈরি করা যায় {} ব্র্যাকেট ব্যবহার করে অথবা set() function দিয়ে।

```
s1 = {10, 20, 30}
s2 = set([1, 2, 3])
print(s1)
print(s2)
```

আউটপুট:

```
{10, 20, 30}
```

{1, 2, 3}
দ্রষ্টব্য: {} ব্যবহার করলে যদি কোনো মান না দেওয়া হয়, Python সেটিকে empty dictionary ধরে নেয়। তাই empty set তৈরি করতে set() ব্যবহার করতে হবে।

```
s = set()
print(type(s))
```
আউটপুট:

```
<class 'set'>
```

Set-এর Elements add ও delete করা
Set mutable হওয়ায় এতে নতুন elements যোগ বা পুরনো মুছে ফেলা যায়।

```
numbers = {1, 2, 3}
numbers.add(4)
print(numbers)

numbers.remove(2)
print(numbers)
```
আউটপুট:

```
{1, 2, 3, 4}
{1, 3, 4}
```
discard() method ব্যবহার করলে element না থাকলেও error দেয় না (কোনো আউটপুট থাকবে না, কারণ আমরা কিছু print করিনি)।

```
numbers.discard(10)
```

Set Operations
Set-এ গণিতের মতো বিভিন্ন operation করা যায়।

Operation Symbol Description
Union ``

Intersection & শুধুমাত্র common elements দেয়

Difference - প্রথম set থেকে দ্বিতীয় set-এর element বাদ দেয়

Symmetric Difference ^ উভয়ের unique elements দেয়
উদাহরণ:

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)
```

```
print(a & b)
print(a - b)
print(a ^ b)
```
আউটপুট:

```
{1, 2, 3, 4, 5}
{3}
{1, 2}
{1, 2, 4, 5}
```

Set Iteration

Set-এর ওপর loop চালানো যায়, কিন্তু যেহেতু এটি unordered, তাই output-এর ক্রম পরিবর্তিত হতে পারে।

```
for item in {10, 20, 30}:
    print(item)
```
সম্ভাব্য আউটপুট (ক্রম ভিন্ন হতে পারে):

```
10
20
30
```

Membership Checking

in এবং not in অপারেটর দিয়ে Set-এর মধ্যে কোনো মান আছে কিনা তা যাচাই করা যায়।

```
s = {1, 2, 3}
print(2 in s)
print(5 not in s)
```
আউটপুট:

```
True
True
```

Set Comprehension

Set comprehension list comprehension-এর মতোই, তবে {} ব্যবহার করা হয়।

```
squares = {x*x for x in range(5)}
print(squares)
```
আউটপুট (ক্রম ভিন্ন হতে পারে):

```
{0, 1, 4, 9, 16}
```

মূল পয়েন্টসমূহ

- Set হলো unordered, mutable এবং unique elements-এর collection।
- Duplicate element স্বয়ংক্রিয়ভাবে বাদ যায়।

- Union, intersection, difference ইত্যাদি operation সমর্থন করে।
- Empty set তৈরি করতে set() ব্যবহার করতে হয়।

Dictionaries in Python

Dictionary কী

Dictionary হলো একটি key-value pair ভিত্তিক data structure, যেখানে প্রতিটি মানের সাথে একটি নির্দিষ্ট key যুক্ত থাকে। এটি unordered হলেও Python 3.7 থেকে insertion ordered হয়ে গেছে।

উদাহরণ:

```
student = {"name": "Rafi", "age": 20, "grade": "A"}
print(student)
```

আউটপুট:

```
{'name': 'Rafi', 'age': 20, 'grade': 'A'}
```

Create a Dictionary

Dictionary তৈরি করতে {} ব্যবহার করা হয় অথবা dict() function দিয়ে তৈরি করা যায়।

```
info1 = {"name": "Amit", "age": 25}
info2 = dict(city="Dhaka", country="Bangladesh")
print(info1)
print(info2)
```

আউটপুট:

```
{'name': 'Amit', 'age': 25}
{'city': 'Dhaka', 'country': 'Bangladesh'}
```

Access elements in a Dictionary

Key ব্যবহার করে Value access করা হয়।

```
person = {"name": "Sara", "age": 22}
print(person["name"])
print(person.get("age"))
```

আউটপুট:

```
Sara
22
```

get() method ব্যবহার করলে key না থাকলে error দেয় না।

```
print(person.get("city", "Not Found"))
```
আউটপুট:

```
Not Found
```

Add or update elements in a Dictionary

Dictionary mutable, তাই নতুন key-value pair যোগ করা যায় বা পুরনো মান পরিবর্তন করা যায়।

```
student = {"name": "Mina", "age": 18}
student["grade"] = "A"
student["age"] = 19
print(student)
```
আউটপুট:

```
{'name': 'Mina', 'age': 19, 'grade': 'A'}
```

Delete elements from a Dictionary

To delete elements, you can use the following methods.

```
student = {"name": "Ali", "age": 20, "grade": "B"}
student.pop("age")
print(student)
```
আউটপুট:

```
{'name': 'Ali', 'grade': 'B'}
```
Using clear() makes the Dictionary empty.

```
student.clear()
print(student)
```
আউটপুট:

```
{}
```

Dictionary Iteration

Dictionary-এর key, value বা key-value pair এর ওপর loop চালানো যায়।

```
person = {"name": "Rafi", "age": 21}

for key in person:
    print(key, ":", person[key])

for value in person.values():
    print(value)
```

```
for key, value in person.items():
    print(key, value)
```
Output

```
name : Rafi
age : 21
Rafi
21
name Rafi
age 21
```

Dictionary Methods

Method	কাজ
keys()	সব key ফেরত দেয়
values()	সব value ফেরত দেয়
items()	key-value pair ফেরত দেয়
pop(key)	নির্দিষ্ট key-value pair মুছে ফেলে
clear()	পুরো Dictionary খালি করে
update()	আরেকটি Dictionary-এর ডেটা যোগ করে

উদাহরণ:

```
a = {"x": 1, "y": 2}
b = {"y": 3, "z": 4}
a.update(b)
print(a)
```
আউটপুট:

```
{'x': 1, 'y': 3, 'z': 4}
```

Nested Dictionary

Dictionary-এর ভেতরেও Dictionary থাকতে পারে।

```
students = {
    "101": {"name": "Rafi", "grade": "A"},
    "102": {"name": "Mina", "grade": "B"}
}
print(students["101"]["name"])
```
আউটপুট:

```
Rafi
```

মূল পয়েন্টসমূহ

- Dictionary হলো key-value pair ভিত্তিক structure।

- Key অবশ্যই unique এবং immutable হতে হয়।
- Dictionary is mutable, so you can update it.
- keys(), values(), items() দিয়ে সহজে iteration করা যায়।

Dictionary Methods in Python

Read (Access) Methods

get() দিয়ে safe read করা যায়; key না থাকলে error নয়, বরং default রিটার্ন হয়।

```
user = {"name": "Nadia", "age": 23}
print(user.get("name"))
print(user.get("city"))
print(user.get("city", "Unknown"))
```

আউটপুট:

```
Nadia
None
Unknown
```

keys(), values(), items() view object দেয় যা dynamic—ডিকশনারি বদলালে view-ও reflect করবে।

```
user = {"name": "Nadia", "age": 23}
print(user.keys())
print(user.values())
print(user.items())
```

আউটপুট (representation ভিন্ন হতে পারে):

```
dict_keys(['name', 'age'])
dict_values(['Nadia', 23])
dict_items([('name', 'age'), ('age', 23)])
```

Create Methods

fromkeys() দিয়ে iterable থেকে keys তৈরি করা যায়, সবগুলোর জন্য same value সেট হয়।

```
keys = ["id", "name", "age"]
defaults = dict.fromkeys(keys, None)
print(defaults)
```

আউটপুট:

```
{'id': None, 'name': None, 'age': None}
```

নোট: যদি mutable object (যেমন list) value হিসেবে দাও, সব key একই object share করবে।

```
fields = dict.fromkeys(["a", "b"], [])
```

```
fields["a"].append(1)
print(fields)
```
আউটপুট:

```
{'a': [1], 'b': [1]}
```

Update/Add Methods

update() অন্য mapping বা key-value pair iterable থেকে merge করে। Existing key থাকলে overwrite হয়।

```
profile = {"name": "Nadia", "age": 23}
profile.update({"age": 24, "city": "Dhaka"})
print(profile)
```
আউটপুট:

```
{'name': 'Nadia', 'age': 24, 'city': 'Dhaka'}
```
setdefault() missing key create করতে সুবিধাজনক; key থাকলে সেটির value-ই রিটার্ন করে।

```
settings = {"theme": "light"}
val = settings.setdefault("lang", "bn")
print(val)
print(settings)
```
আউটপুট:

```
bn
{'theme': 'light', 'lang': 'bn'}
```

Delete Methods

pop() নির্দিষ্ট key remove করে এবং value রিটার্ন করে। Default দিলে missing key হলে default রিটার্ন করে।

```
stock = {"apple": 5, "banana": 10}
q = stock.pop("apple")
print(q)
print(stock)

missing = stock.pop("orange", 0)
print(missing)
print(stock)
```
আউটপুট:

```
5
{'banana': 10}
0
{'banana': 10}
```

popitem() last inserted pair remove করে (Python 3.7+)।

```
pairs = {"x": 1, "y": 2, "z": 3}
item = pairs.popitem()
print(item)
print(pairs)
```

আউটপুট (LIFO):

```
('z', 3)
{'x': 1, 'y': 2}
```

Empty & Copy

clear() পুরো dictionary empty করে দেয়।

```
data = {"a": 1, "b": 2}
data.clear()
print(data)
```

আউটপুট:

```
{}
```

copy() shallow copy রিটার্ন করে—nested mutable object শেয়ার্ড থাকতে পারে।

```
orig = {"nums": [1, 2]}
cloned = orig.copy()
cloned["nums"].append(3)
print(orig)
print(cloned)
```

আউটপুট:

```
{'nums': [1, 2, 3]}
{'nums': [1, 2, 3]}
```

Deep copy দরকার হলে copy মডিউলের deepcopy() ব্যবহার করো।

Performance Notes (সংক্ষিপ্ত)

- Most dict ops average O(1) (hash table ভিত্তিক)।
- keys()/values()/items() view অবজেক্ট—iterate করতে O(n)।

মূল পয়েন্টসমূহ

- Safe read-এর জন্য get() এবং create-if-missing-এর জন্য setdefault()।
- Merge/overwrite করতে update()।
- নির্দিষ্ট remove-এর জন্য pop(); last-in remove-এর জন্য popitem()।
- সবকিছু empty করতে clear(); shallow copy() nested mutable share করে।

Dictionary View Methods

What are dictionary views?

keys(), values(), items() মেথডগুলো view object রিটার্ন করে—এগুলো lightweight এবং ডিকশনারি পরিবর্তিত হলে view-ও সাথে সাথে reflect করে।

```
user = {"name": "Nadia", "age": 23}
ks = user.keys()
vs = user.values()
its = user.items()
print(ks)
print(vs)
print(its)
```

আউটপুট (representation ভিন্ন হতে পারে):

```
dict_keys(['name', 'age'])
dict_values(['Nadia', 23])
dict_items([('name', 'Nadia'), ('age', 23)])
```

Views are dynamic (live)

ডিকশনারিতে add/update/delete করলে view সঙ্গে সঙ্গে reflect করে।

```
user = {"name": "Nadia"}
ks = user.keys()
print(list(ks))

user["age"] = 23  # add
print(list(ks))    # ks আবার নতুন key দেখাবে

user.pop("name")  # delete
print(list(ks))
```

আউটপুট:

```
['name']
['name', 'age']
['age']
```

Iterate and membership with views

Read/iterate করা ও membership check করা সহজ।

```
person = {"id": 101, "city": "Dhaka"}
for k in person.keys():
    print(k)
```

```
for v in person.values():
    print(v)
for k, v in person.items():
    print(k, v)

print("city" in person.keys())   # membership on keys
print("Khulna" in person.values())
```
সম্ভাব্য আউটপুট:

```
id
city
101
Dhaka
id 101
city Dhaka
True
False
```

Convert views to concrete containers

কখনো স্থির snapshot দরকার হলে view-কে list/tuple/set-এ convert করা হয় (যেমন sorting/Indexing দরকার হলে)।

```
data = {"b": 2, "a": 1, "c": 3}
key_list = list(data.keys())
value_tuple = tuple(data.values())
key_set = set(data.keys())
print(key_list)
print(value_tuple)
print(key_set)
```
আউটপুট (ক্রম ভিন্ন হতে পারে):

```
['b', 'a', 'c']
(2, 1, 3)
{'a', 'b', 'c'}
```
নোট: list/tuple/set বানালে সেটা আর live থাকে না—এটি একটি snapshot।

Set-like operations on views (keys/items)

dict_keys এবং dict_items view গুলো set-like—এগুলোতে অন্য set বা keys view-এর সাথে & (intersection), | (union), - (difference) করা যায়।

```
p = {"id": 1, "name": "Ana", "city": "Dhaka"}
q = {"name": "Ana", "country": "BD"}

common_keys = p.keys() & q.keys()
```

```
all_keys = p.keys() | q.keys()
left_only = p.keys() - q.keys()
print(common_keys)
print(all_keys)
print(left_only)

# items()-এর ক্ষেত্রেও
common_items = p.items() & {("name", "Ana"), ("id", 1)}
print(common_items)
```
আউটপুট (ক্রম ভিন্ন হতে পারে):

```
{'name'}
{'id', 'name', 'city', 'country'}
{'city', 'id'}
{('id', 1), ('name', 'Ana')}
```
নোট: dict_values view set-like অপারেশনের জন্য উপযুক্ত নয়; দরকার হলে আগে set() নাও (values hashable হলে)।

Sorting or indexing with views
View নিজে index/sort সাপোর্ট করে না। Indexing/Sorting দরকার হলে list-এ convert করো।

```
scores = {"Rafi": 88, "Mina": 92, "Ali": 85}

# Top scorer (by value) বের করতে values দিয়ে sort
top = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)[0]
print(top)
```
আউটপুট:

```
('Mina', 92)
```

Performance notes (সংক্ষিপ্ত)
- Views memory-efficient এবং তৈরি করা O(1)।
- Iterate করা O(n)। Snapshot (list/tuple/set) বানালে O(n) কপি খরচ হয়।

মূল পয়েন্টসমূহ
- keys()/values()/items() view object—live update reflect করে।
- Membership/iteration/read-এর জন্য views যথেষ্ট; স্থির কপি দরকার হলে list/tuple/set নাও।
- Keys/items view-এ set-like operations কাজ করে; values-এর ক্ষেত্রে আগে set() লাগতে পারে।

Dictionary Comprehension

Dictionary Comprehension কী

Dictionary comprehension হলো সংক্ষিপ্ত সিনট্যাক্স ব্যবহার করে নতুন dictionary create করার পদ্ধতি।
সাধারণ ফর্ম:

{key_expr: value_expr for item in iterable if condition}

Create: বেসিক উদাহরণ

সংখ্যা 1..5 এর square values দিয়ে dict create করা:

squares = {x: x*x for x in range(1, 6)}
print(squares)
আউটপুট:

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Read/Filter: শর্ত দিয়ে নির্বাচন

শুধু even key রেখে dict create করা (read/select-স্টাইল):

evens = {x: x*x for x in range(1, 8) if x % 2 == 0}
print(evens)
আউটপুট:

{2: 4, 4: 16, 6: 36}
নোট: if শর্তটি for-এর পরে দিলে item ফিল্টার হয়; value-তে if-else দিলে মান পরিবর্তন হয়।

Transform/Update-style: মান পরিবর্তন

কমপ্রিহেনশন দিয়ে value transform করা (যেমন, string normalize):

raw = {"Name": "Rafi", "CITY": "Dhaka"}
normalized = {k.lower(): v.upper() for k, v in raw.items()}
print(normalized)
আউটপুট:

{'name': 'RAFI', 'city': 'DHAKA'}

Conditional transform (ternary):

nums = {1: 10, 2: 15, 3: 20}
flagged = {k: ("EVEN" if v % 2 == 0 else "ODD") for k, v in nums.items()}
print(flagged)

আউটপুট:

{1: 'EVEN', 2: 'ODD', 3: 'EVEN'}

তালিকা/টিউপল থেকে **dict create**

Tuple list → dict:

```
pairs = [("a", 1), ("b", 2), ("c", 3)]
D = {k: v for k, v in pairs}
print(D)
```

আউটপুট:

{'a': 1, 'b': 2, 'c': 3}

দুটি আলাদা list → zip() দিয়ে dict:

```
keys = ["id", "name", "age"]
vals = [101, "Mina", 20]
info = {k: v for k, v in zip(keys, vals)}
print(info)
```

আউটপুট:

{'id': 101, 'name': 'Mina', 'age': 20}

Invert: key ↔ value বদলানো

সতর্কতা: values অবশ্যই hashable হতে হবে এবং duplicate value হলে শেষটি থাকবে।

```
orig = {"x": 1, "y": 2, "z": 1}
inverted = {v: k for k, v in orig.items()}
print(inverted)
```

আউটপুট:

{1: 'z', 2: 'y'}

Nested Comprehension

Nested iterable থেকে flat dict create করা:

```
matrix = [[1, 2], [3, 4]]
flat = {idx: val for row in matrix for idx, val in enumerate(row, 1)}
print(flat)
```

আউটপুট:

{1: 2, 2: 4}

আরও স্পষ্ট উদাহরণ (pair numbering):

```
pairs = [[("a", 1), ("b", 2)], [("c", 3)]]
flat_pairs = {k: v for sub in pairs for (k, v) in sub}
```

print(flat_pairs)
আউটপুট:

{'a': 1, 'b': 2, 'c': 3}

Comprehension vs for-loop (equivalent)

কমপ্রিহেনশনের সমতুল্য লুপ কোড:

```
# comprehension
squares = {x: x*x for x in range(3)}

# equivalent for-loop
squares2 = {}
for x in range(3):
    squares2[x] = x*x

print(squares)
print(squares2)
```

আউটপুট:

{0: 0, 1: 1, 2: 4}
{0: 0, 1: 1, 2: 4}

Common Pitfalls

- Duplicate keys overwrite: একই key একাধিকবার এলে শেষ মানটি থাকবে।
- Mutable shared value: যদি একই mutable object রেফারেন্স দাও, সব key তাতে share করবে।
- Complex logic: অতিরিক্ত nested/complex হলে পড়া কঠিন—লুপ ব্যবহার করো।
- Order: Python 3.7+ এ dict insertion ordered; comprehension-ও সে অনুসারেই order রাখে।

Performance Notes

- comprehension সাধারণত সমতুল্য লুপের চেয়ে concise এবং সামান্য দ্রুত।
- অতিরিক্ত complex condition/নেস্টিং performance/পঠনযোগ্যতা কমাতে পারে।

মূল পয়েন্টসমূহ

- {k: v for ...} দিয়ে dict দ্রুত create করা যায়।
- if দিয়ে filter করা যায়; if-else দিয়ে transform করা যায়।
- zip()/list-of-tuples থেকে dict বানানো সহজ।
- Duplicate keys হলে শেষ মানটি থাকবে; value hashable না হলে invert সম্ভব নয়।

Intro to Function

ফাংশন কী?
ফাংশন হলো কোডের একটি অংশ যা শুধু তখনই চলে যখন তুমি তাকে call করো। এটা কোড বারবার ব্যবহার করতে সাহায্য করে।

ফাংশন লেখার নিয়ম

```
def greet():
    print("hello")
```

def → ফাংশন লেখার কীওয়ার্ড
greet() → ফাংশনের নাম

ফাংশন কল করা
```
greet()
```

আউটপুট:

```
hello
```

কেন ফাংশন ব্যবহার করব?
কোড পরিষ্কার ও সংগঠিত রাখে
কোড পুনরায় ব্যবহার করা যায়
ডিবাগ ও পরিবর্তন সহজ হয়
উদাহরণ

```
def greet():
    print("hello")

greet()   # ফাংশন কল করা
```
আউটপুট: hello

Function Parameters

Parameter কী?
Parameter হলো ফাংশনের ডিক্লেয়ারেশনে ব্যবহৃত ভেরিয়েবল।

Argument হলো ফাংশন কল করার সময় প্যারামিটারে দেওয়া মান।

Default Parameter

```
def greet(user="guest"):
    print(f"hello {user}")

greet("adil")  # Output: hello adil
greet()        # Output: hello guest
```

user="guest" হলো ডিফল্ট প্যারামিটার → কোনো আর্গুমেন্ট না দিলে ডিফল্টভাবে "guest" ব্যবহার হবে।

Positional Argument

```
def square_addition(a, b, c):
    print(f"{a} {b} {c}")
    a = a**2
    b = b**2
    return a + b + c

ans = square_addition(10, 20, 30) + 10
print(ans)  # Output: 540
```

মানগুলো ক্রম অনুযায়ী যায় → a=10, b=20, c=30

Keyword Argument

```
ans = square_addition(b=10, c=20, a=30) + 10
print(ans)  # Output: 1030
```

মানগুলো key অনুযায়ী দেওয়া হয়েছে, ক্রম position অনুযায়ী নয়।

Arbitrary Arguments (args)

কতগুলো positional argument আসবে তা আগে থেকে জানা না থাকলে *args ব্যবহার করা হয়।

```
def square_addition(*args):
    summation = 0
    for i in args:
        summation += i**2
    return summation
```

```
ans = square_addition(10, 20, 30, 40, 50)
print(ans)  # Output: 5500
```

যত মানই দাও না কেন, সবগুলোর square করে যোগ হবে।

Keyword Arguments Dictionary (**kwargs)

ইনপুট নামসহ (key=value আকারে) দেওয়া হলে এবং সংখ্যা অজানা থাকলে **kwargs ব্যবহার করা হয়।

```
def student(**kwargs):
    for key, val in kwargs.items():
        print(f"{key} : {val}")

student(name="adil", cls=10, roll=30, marks=30.5)
```

আউটপুট:

```
name : adil
cls : 10
roll : 30
marks : 30.5
```

**kwargs সব নাম এবং মান জোড়াকে dictionary আকারে নেয়।

Function Return Type

return কী?

return ফাংশনের কাজ শেষ হওয়ার পর একটা বা একাধিক মান বাইরে পাঠায়।

এই মান পরে আমরা ব্যবহার করতে পারি বা কোনো ভ্যারিয়েবলে সংরক্ষণ করতে পারি।

একটি মান রিটার্ন করা

```
def give_prediction():
    return 10

print(type(give_prediction()))
```

আউটপুট: <class 'int'>

ফাংশনটি একটি ইন্টিজার মান রিটার্ন করেছে।

ভিন্ন ডাটা টাইপ রিটার্ন করা

```
def give_prediction():
   return 10.5
print(type(give_prediction()))  # Output: <class 'float'>

def give_prediction():
   return "name"
print(type(give_prediction()))  # Output: <class 'str'>
```

একাধিক মান রিটার্ন করা

```
def give_prediction(a, b):
   return a, b

print(type(give_prediction(10,30)))  # Output: <class 'tuple'>
print(give_prediction(10,30))        # Output: (10, 30)
```

যখন একাধিক মান রিটার্ন করা হয়, Python tuple আকারে সেগুলো সংরক্ষণ করে।

Unpacking

```
x, y = give_prediction(10,30)
print(x, y)  # Output: 10 30
```

এখানে x=10, y=30 হয়েছে , এখানে Tuple এর মান আলাদা করে নেওয়া হয়েছে।

লিস্ট রিটার্ন করা

```
def give_prediction():
   a = [10,20,30]
   b = [20,30,40]
   return a, b

x, y = give_prediction()
print(x)  # Output: [10, 20, 30]
print(y)  # Output: [20, 30, 40]
```

সেট ও লিস্ট রিটার্ন করা

```
def give_prediction():
    a = {10,20,30}
    b = [100,200,300]
    return a, b

x = give_prediction()
print(type(x))  # Output: <class 'tuple'>

for item in x:
    print(item)
```

আউটপুট:

```
<class 'tuple'>
{10, 20, 30}
[100, 200, 300]
```

সেট {} এবং লিস্ট [] একসাথে রিটার্ন করলে Python Tuple এ মধ্যে রেখে দেয়।

ডিকশনারি রিটার্ন করা

```
def give_prediction():
    return {"name": "adil", "addr": "Ctg"}, {"name": "abrar", "addr": "dhaka"}

x, y = give_prediction()
print(type(x))  # Output: <class 'dict'>

for item in x.items():
    print(item)
```

আউটপুট:

```
<class 'dict'>
('name', 'adil')
('addr', 'Ctg')
```

.items() ব্যবহার করলে dictionary থেকে key–value pair পাওয়া যায়।

Iterator and Generator

Iterator কী?

Iterator হলো একটি Python অবজেক্ট যা দিয়ে আমরা step by step traverse করতে পারি। এটি iter() এবং next() মেথড ব্যবহার করে কাজ করে।

Iterator এর বৈশিষ্ট্য:

iter() দিয়ে তৈরি হয়
next() দিয়ে step by step মান নেওয়া যায়
সব মান শেষ হয়ে গেলে StopIteration error দেয়

Iterator উদাহরণ

```
s = {10, 20, 30, 40, 50}

s_iter = iter(s)      # set থেকে iterator তৈরি
print(next(s_iter))   # প্রথম মান
print(next(s_iter))   # দ্বিতীয় মান
print(next(s_iter))
print(next(s_iter))
print(next(s_iter))
```

আউটপুট (set unordered তাই order ভিন্ন হতে পারে):

```
40
10
50
20
30
```

for loop এর ভেতরের কাজ:

```
s = {10, 20, 30, 40, 50}

for i in s:
    print(i)
```

ভেতরে যা ঘটে:

```
s_iter = iter(s)
while True:
    try:
        i = next(s_iter)
        print(i)
    except StopIteration:
        break
```

Generator কী**?**

Generator হলো একটি বিশেষ ধরনের ফাংশন যা yield কীওয়ার্ড ব্যবহার করে। এটি মেমরি-এফিশিয়েন্টভাবে কাজ করে কারণ সব ভ্যালু একসাথে মেমরিতে রাখে না।

Generator এর বৈশিষ্ট্য**:**

yield ব্যবহার করে
একবারে একটি মান রিটার্ন করে
State রক্ষা করে → পরের বার yield আগের yield এর পর থেকে শুরু হয়
মেমরি-এফিশিয়েন্ট

Generator উদাহরণ → ১

```
def simple_generator():
    yield 1
    yield 2
    yield 3

gen = simple_generator()
print(next(gen))  # Output: 1
print(next(gen))  # Output: 2
print(next(gen))  # Output: 3
```

Generator উদাহরণ → ২

```
lst = [x for x in range(500)]   # বড় dataset

def data_loader(chunk_size, lst):
    for i in range(0, len(lst), chunk_size):
        yield lst[i:i+chunk_size]  # ছোট ছোট অংশ রিটার্ন করে
```

```
# ব্যবহার
x = data_loader(5, lst)
print(next(x))  # Output: [0, 1, 2, 3, 4]
print(next(x))  # Output: [5, 6, 7, 8, 9]
print(next(x))  # Output: [10, 11, 12, 13, 14]
```

প্রতিবার next() কল করলে পরবর্তী ৫টি আইটেমের লিস্ট রিটার্ন হয়।

Generator Expression

```
# List Comprehension -> মেমরিতে সব ভ্যালু রাখে
list_comp = [x*x for x in range(5)]  # [0, 1, 4, 9, 16]

# Generator Expression -> মেমরিতে শুধু প্রয়োজনে ভ্যালু তৈরি করে
gen_exp = (x*x for x in range(5))

print(list_comp)  # Output: [0, 1, 4, 9, 16]
print(next(gen_exp))  # Output: 0
print(next(gen_exp))  # Output: 1
```

কখন **Generator** ব্যবহার করব**?**

বড় ডেটাসেট নিয়ে কাজ করতে
মেমরি সাশ্রয় করতে
Infinite sequence তৈরি করতে
পাইপলাইন তৈরি করতে → একটি জেনারেটর থেকে অন্যটিতে ডেটা পাঠানো

Infinite Sequence উদাহরণ

```
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1

gen = infinite_sequence()
print(next(gen))  # 0
print(next(gen))  # 1
print(next(gen))  # 2
```

... চলতেই থাকবে

Lambda function in python

Lambda Function কী?

Lambda function হলো Python-এর একটি বিশেষ ফাংশন যা:

Anonymous → আলাদা নাম দেওয়ার প্রয়োজন নেই
এক লাইনের → সংক্ষিপ্তভাবে লেখা যায়
হালকা ও দ্রুত → সাধারণ ফাংশনের চেয়ে কম মেমরি ব্যবহার করে এবং দ্রুত Execution হয়ে থাকে
একে Anonymous Function ও বলা হয়।

সিনট্যাক্স:

lambda arguments: expression

সাধারণ ফাংশন **vs Lambda** ফাংশন

সাধারণ ফাংশন:

```
def square_addition(x, y):
    return x**2 + y**2

result = square_addition(2, 3)
print(result)  # Output: 13
```

Lambda ফাংশন:

```
sq_add = lambda x, y: x**2 + y**2
print(sq_add(2, 3))  # Output: 13
```

Lambda ফাংশনের বিভিন্ন উদাহরণ

দুটি সংখ্যার যোগ

```
add = lambda a, b: a + b
print(add(5, 10))  # Output: 15
```

Square করা

```
square = lambda n: n**2
print(square(4))  # Output: 16
```

স্ট্রিং দিয়ে

```
greet = lambda name: f"Hello {name}"
print(greet("Adil"))  # Output: Hello Adil
```

শর্তসহ Lambda ফাংশন

```
check_even = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check_even(5))  # Output: Odd
print(check_even(8))  # Output: Even
```

Lambda ফাংশনের ব্যবহার**:**

সরাসরি কল করা:

```
result = (lambda x, y: x * y)(5, 3)
print(result)  # Output: 15
```

লিস্টের ভিতরে:

```
operations = [
    lambda x: x + 1,
    lambda x: x * 2,
    lambda x: x ** 2
]

for func in operations:
    print(func(5))  # Output: 6, 10, 25
```

ডিকশনারির ভিতরে:

```
math_ops = {
    'add': lambda x, y: x + y,
    'multiply': lambda x, y: x * y,
    'power': lambda x, y: x ** y
}

print(math_ops['add'](10, 5))       # Output: 15
print(math_ops['power'](2, 3))      # Output: 8
```

বাস্তব ব্যবহারের উদাহরণ:

```
# Grade Checker
grade_check = lambda marks: "A+" if marks >= 80 else "A" if marks >= 70 else "B" if marks >= 60 else "C"
print(grade_check(85))  # Output: A+
print(grade_check(65))  # Output: B

# Number Checker
number_check = lambda x: (
   "Positive" if x > 0 else
   "Negative" if x < 0 else
   "Zero"
)
print(number_check(10))   # Output: Positive
print(number_check(-5))   # Output: Negative
print(number_check(0))    # Output: Zero
```

সীমাবদ্ধতা:

- শুধু এক লাইনে লিখতে হয়
- জটিল logic লেখা যায় না
- একাধিক statement support করে না
- Debug করা কঠিন

Map Function in Python

map() ফাংশন কী?

map() হলো Python-এর একটি বিল্ট-ইন ফাংশন যা একটি ফাংশন এবং একটি iterable (যেমন: list, tuple) নেয় এবং সেই ফাংশনটিকে iterable-এর প্রতিটি উপাদানে প্রয়োগ করে।

সিনট্যাক্স:

```
map(function, iterable)
```

প্যারামিটার:

function → যে ফাংশনটি প্রতিটি উপাদানে প্রয়োগ করব
iterable → যেকোনো iterable অবজেক্ট (list, tuple, ইত্যাদি)

রিটার্ন:

একটি special iterator object রিটার্ন করে ,যা সরাসরি প্রিন্ট করা যায় না, list-এ convert করতে হয়

উদাহরণ ১ → **Normal** ফাংশনের সাথে **map()**

```
# Sqaure করা
numbers = [1, 2, 3, 4, 5, 6]

def square(x):
    return x ** 2

result = map(square, numbers)
print(list(result))
```

আউটপুট:

[1, 4, 9, 16, 25, 36]

square() ফাংশনটি numbers লিস্টের প্রতিটি সংখ্যায় প্রয়োগ হয়েছে।

উদাহরণ ২ → বিল্ট-ইন ফাংশনের সাথে **map()**

```
# স্ট্রিং-এর অক্ষরগুলোকে uppercase করা
text = "hello world"

result = map(str.upper, text)
uppercase_list = list(result)
print(uppercase_list)

# স্ট্রিং হিসেবে পেতে
uppercase_text = ''.join(uppercase_list)
print(uppercase_text)
```

আউটপুট:

```
['H', 'E', 'L', 'L', 'O', ' ', 'W', 'O', 'R', 'L', 'D']
HELLO WORLD
```

উদাহরণ ৩ → **Lambda** ফাংশনের সাথে **map()**

```
# Lambda ব্যবহার করে বর্গ করা
numbers = [1, 2, 3, 4, 5, 6]

result = map(lambda x: x ** 2, numbers)
print(list(result))
```

আউটপুট:

```
[1, 4, 9, 16, 25, 36]
```

Lambda ফাংশন দিয়ে আলাদা ফাংশন ডিফাইন না করেই কাজ করা যায়।

উদাহরণ ৪ → একাধিক **Iterable** এর সাথে **map()**

```
# দুটি লিস্টের সংখ্যা যোগ করা
list1 = [1, 2, 3, 4]
list2 = [10, 20, 30, 40]

result = map(lambda x, y: x + y, list1, list2)
print(list(result))
```

আউটপুট:

```
[11, 22, 33, 44]
```

Lambda ফাংশন দুটি আর্গুমেন্ট নিচ্ছে এবং প্রতিটি পজিশনের সংখ্যা যোগ করছে।

উদাহরণ ৫ → বিভিন্ন ডেটা টাইপ কনভার্সন

```
# স্ট্রিং থেকে ইন্টিজারে কনভার্ট
string_numbers = ["1", "2", "3", "4", "5"]

result = map(int, string_numbers)
numbers_list = list(result)
print(numbers_list)
print(type(numbers_list[0]))
```

আউটপুট:

```
[1, 2, 3, 4, 5]
<class 'int'>
```

map() ফাংশনের কাজের পদ্ধতি**:**

```
# Manual ভাবে যা হয়:
def manual_map(function, iterable):
    result = []
    for item in iterable:
        result.append(function(item))
    return result

# ব্যবহার
numbers = [1, 2, 3]
squared = manual_map(lambda x: x**2, numbers)
print(squared)  # Output: [1, 4, 9]
```

map() vs List Comprehension:

```
map() ব্যবহার:
numbers = [1, 2, 3, 4]
result = map(lambda x: x * 2, numbers)
print(list(result))  # Output: [2, 4, 6, 8]


List Comprehension ব্যবহার:
numbers = [1, 2, 3, 4]
result = [x * 2 for x in numbers]
print(result)  # Output: [2, 4, 6, 8]
```

তুলনা:

- map() - Functional programming style
- List Comprehension - Pythonic style
- Performance প্রায় একই

কখন map() ব্যবহার করব?

- সব উপাদানে একই অপারেশন প্রয়োগ করতে
- Functional programming style এ কোড লিখতে
- Memory efficient কোড লিখতে → lazy evaluation
- Built-in functions সহ ব্যবহার করতে

Filter Function in Python

filter() ফাংশন কী?

filter() হলো Python-এর একটি বিল্ট-ইন ফাংশন, যা একটি ফাংশন এবং একটি iterable (যেমন list, tuple ইত্যাদি) নেয়, এবং শুধুমাত্র সেই উপাদানগুলো রিটার্ন করে যেগুলোর জন্য ফাংশনটি True রিটার্ন করে।

সিনট্যাক্স:

filter(function, iterable)
প্যারামিটার:

function → এমন একটি ফাংশন যা প্রতিটি উপাদানের জন্য True বা False রিটার্ন করবে
iterable → যেমন list, tuple, string ইত্যাদি

রিটার্ন:

একটি filter object → যেটাকে সাধারণত list() দিয়ে কনভার্ট করা হয়

উদাহরণ ১ → জোড় সংখ্যা বের করা

```
numbers = list(range(20))

# Lambda function ব্যবহার করে
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))
```

আউটপুট:

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

এখানে lambda x: x % 2 == 0 শুধু সেই সংখ্যাগুলোর জন্য True রিটার্ন করছে যেগুলো জোড়।

উদাহরণ ২ → সাধারণ ফাংশনের সাথে **filter()**

```
def is_positive(number):
    return number > 0

numbers = [-5, -2, 0, 3, 8, -1, 10]
```

```
positive_numbers = filter(is_positive, numbers)
print(list(positive_numbers))
```
আউটপুট:

```
[3, 8, 10]
```

উদাহরণ ৩ → **Truthy Values** ফিল্টার করা

```
data = [0, 1, '', 'Hello', None, [], [1, 2], False, True]

# পদ্ধতি ১: filter(None, data)
cleaned_data = list(filter(None, data))
print("filter(None):", cleaned_data)

# পদ্ধতি ২: Lambda দিয়ে চেক করা
non_empty_data = list(filter(lambda x: x != '' and x is not None, data))
print("Lambda:", non_empty_data)
```
আউটপুট:

```
filter(None): [1, 'Hello', [1, 2], True]
Lambda: [0, 1, 'Hello', [], [1, 2], False, True]
```

filter(None, data) শুধুমাত্র truthy values রাখে → যেগুলো boolean হিসেবে True হয়।

উদাহরণ ৪ → স্ট্রিং ফিল্টার করা

```
text = "Hello World! Welcome to Python Programming."

# শুধুমাত্র বড় হাতের অক্ষর
uppercase_letters = list(filter(str.isupper, text))
print("Uppercase:", uppercase_letters)

# শুধুমাত্র অক্ষর (alphabet)
alphabetic_chars = list(filter(str.isalpha, text))
print("Alphabetic:", alphabetic_chars)
```

আউটপুট:

```
Uppercase: ['H', 'W', 'P']
Alphabetic: ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', 'W', 'e', 'l', 'c', 'o', 'm', 'e', 't', 'o', 'P', 'y', 't', 'h', 'o', 'n', 'P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g']
```

উদাহরণ ৫ → জটিল শর্তসহ ফিল্টারিং

```
students = [
   {'name': 'Alice', 'age': 20, 'grade': 'A'},
   {'name': 'Bob', 'age': 22, 'grade': 'B'},
   {'name': 'Charlie', 'age': 19, 'grade': 'A'},
   {'name': 'David', 'age': 21, 'grade': 'C'}
]

# 'A' গ্রেড এবং বয়স < ২১
top_young_students = filter(lambda s: s['grade'] == 'A' and s['age'] < 21, students)
print(list(top_young_students))
```

আউটপুট:

```
[{'name': 'Alice', 'age': 20, 'grade': 'A'}, {'name': 'Charlie', 'age': 19, 'grade': 'A'}]
```

filter() vs List Comprehension

filter() ব্যবহার:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))  # [2, 4, 6]
```

List Comprehension ব্যবহার:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = [x for x in numbers if x % 2 == 0]
print(even_numbers)  # [2, 4, 6]
```

filter() → Functional style, সাধারণ condition-এর জন্য ভালো
List comprehension→ Pythonic ও flexible, জটিল condition-এর জন্য ভালো

বাস্তব উদাহরণ

```
# Email validation
emails =
["userexample.com","invalid-email","test@gmail.com","another@domain.org","wrong-format"]

valid_emails = filter(lambda email: '@' in email and '.' in email, emails)
print(list(valid_emails))
# ['user@example.com', 'test@gmail.com', 'another@domain.org']
```

```
# Price filtering
products = [
{'name': 'Laptop', 'price': 1000},
{'name': 'Mouse', 'price': 25},
{'name': 'Keyboard', 'price': 80},
{'name': 'Monitor', 'price': 300}
]

affordable = filter(lambda p: p['price'] <= 100, products)
print(list(affordable))
# [{'name': 'Mouse', 'price': 25}, {'name': 'Keyboard', 'price': 80}]
```
কখন ব্যবহার করব?

- সাধারণ শর্তে ডেটা ফিল্টার করতে
- Functional programming style রাখতে
- Built-in ফাংশন (str.isupper, str.isdigit) ব্যবহার করতে
- Memory-efficient ভাবে কাজ করতে

Reduce Function in Python

reduce() ফাংশন কী?

reduce() হলো Python-এর functools মডিউলের একটি ফাংশন যা একটি iterable (list, tuple, etc.) এর সব উপাদানের উপর ক্রমান্বয়ে একটি ফাংশন প্রয়োগ করে এবং শেষে একটি মাত্র মান রিটার্ন করে।

import করতে হয়:

```
from functools import reduce
```

সিনট্যাক্স:

```
reduce(function, iterable[, initializer])
```

প্যারামিটার:

function → যে ফাংশনটি প্রতিটি ধাপে call হবে (২টি argument নেয়)
iterable → যেকোনো iterable অবজেক্ট
initializer → (optional) initialize মান

উদাহরণ ১ → সংখ্যার যোগফল

```
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]

# Lambda function ব্যবহার করে
summation = reduce(lambda x, y: x + y, numbers)
print(summation)  # Output: 21
```

Step:

Step 1: (1 + 2) → 3
Step 2: (3 + 3) → 6
Step 3: (6 + 4) → 10
Step 4: (10 + 5) → 15
Step 5: (15 + 6) → 21
Final Result: 21

উদাহরণ ২ → সর্বোচ্চ মান বের করা

```
numbers = [10, 5, 25, 8, 30, 15]

max_value = reduce(lambda x, y: x if x > y else y, numbers)
print(max_value)  # Output: 30
```

Step:

Step 1: max(10, 5) → 10
Step 2: max(10, 25) → 25
Step 3: max(25, 8) → 25
Step 4: max(25, 30) → 30
Step 5: max(30, 15) → 30
Final Result: 30

উদাহরণ ৩ → **Initializer** সহ ব্যবহার

```
numbers = [1, 2, 3, 4]

# Initializer 10 দিয়ে শুরু
result = reduce(lambda x, y: x + y, numbers, 10)
print(result)  # Output: 20 (10 + 1 + 2 + 3 + 4)
```

Initializer থাকলে প্রথম ধাপেই initializer এবং প্রথম element নেয়।

উদাহরণ ৪ → স্ট্রিং **concatenation**

```
words = ["Hello", "World", "Python", "Programming"]

sentence = reduce(lambda x, y: x + " " + y, words)
print(sentence)  # Output: Hello World Python Programming
```

উদাহরণ ৫ → সংখ্যার গুণফল

```
numbers = [1, 2, 3, 4, 5]

product = reduce(lambda x, y: x * y, numbers)
print(product)  # Output: 120
```

উদাহরণ ৬ → **List of lists** থেকে সবচেয়ে বড় **list** বের করা

```
lists = [[1, 2], [3, 4, 5], [6], [7, 8, 9, 10]]

longest_list = reduce(lambda x, y: x if len(x) > len(y) else y, lists)
print(longest_list)  # Output: [7, 8, 9, 10]
```

reduce() Step by Step:

```
# Manual reduce implementation
def manual_reduce(func, iterable, initial=None):
    iterator = iter(iterable)

    if initial is None:
        value = next(iterator)
    else:
        value = initial

    for item in iterator:
        value = func(value, item)

    return value

# ব্যবহার
numbers = [1, 2, 3, 4]
```

```
result = manual_reduce(lambda x, y: x + y, numbers)
print(result)  # Output: 10
```

map(), filter(), reduce() তুলনা**:**

map() → Transformation

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # [1, 4, 9, 16]
```

filter() → Selection

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6]
```

reduce() → Aggregation

```
from functools import reduce
numbers = [1, 2, 3, 4]
total = reduce(lambda x, y: x + y, numbers)
print(total)  # 10
```

বাস্তব ব্যবহারের উদাহরণ**:**

গড় বের করা:

```
from functools import reduce
numbers = [10, 20, 30, 40, 50]

# reduce দিয়ে যোগফল, তারপর গড়
total = reduce(lambda x, y: x + y, numbers)
average = total / len(numbers)
print(f"Total: {total}, Average: {average}")
# Output: Total: 150, Average: 30.0
```

ডেটা এগ্রিগেশন:

```
from functools import reduce
sales = [
    {'product': 'A', 'amount': 100},
    {'product': 'B', 'amount': 200},
    {'product': 'A', 'amount': 150},
    {'product': 'C', 'amount': 300}
]
```

```
# Product A-এর মোট sales
total_sales_a = reduce(
    lambda total, sale: total + sale['amount'] if sale['product'] == 'A' else total,
    sales,
    0  # initial value
)
print(f"Total sales for product A: {total_sales_a}")
# Output: Total sales for product A: 250
```

কখন reduce() ব্যবহার করব?

- Aggregation operations (sum, product, etc.)
- Custom aggregation যখন built-in functions যথেষ্ট না
- Functional programming style এ কোড লিখতে
- Complex reductions যখন multiple steps প্রয়োজন

File Handling and Exception Handling

File Handling Basics

File Handling কী

File Handling হলো এমন একটি প্রক্রিয়া যার মাধ্যমে আমরা Python প্রোগ্রাম থেকে file read, write, modify করতে পারি। এটি data সংরক্ষণের জন্য একটি গুরুত্বপূর্ণ বিষয়।

উদাহরণ:

```
f = open('data.txt', 'w')
f.write('Hello Python!')
f.close()
```

আউটপুট:

(data.txt ফাইলে এই টেক্সট লেখা হবে)

File তৈরি ও খোলা (Creating and Opening File)

Python-এ ফাইল খোলার জন্য open() function ব্যবহার করা হয়।

```
file = open(filename, mode, encoding)
```

Parameters
filename: ফাইলের নাম বা path
mode: কীভাবে ফাইল খুলবেন (read, write, append ইত্যাদি)

encoding: টেক্সট ফাইল হলে সাধারণত 'utf-8'
উদাহরণ:

```
f = open('notes.txt', 'r', encoding='utf-8')
```

File Opening Modes

Mode Description
'r' Read mode — ফাইল না থাকলে error দেয়
'w' Write mode — পুরনো কন্টেন্ট মুছে ফেলে নতুন লেখে
'a' Append mode — পুরনো কন্টেন্ট রেখে শেষে যোগ করে
'x' Create new file — ফাইল থাকলে error দেয়
'b' Binary mode
't' Text mode (default)
'+' Update mode (read + write একসাথে)
উদাহরণ:

```
f = open('log.txt', 'a', encoding='utf-8')
```

Empty File তৈরি করা

{} ব্যবহার করলে Python সেটিকে dictionary হিসেবে ধরে নেয়। তাই empty set-এর মতো empty file তৈরির জন্য set() নয়, বরং open() ব্যবহার করতে হয়।

```
f = open('empty.txt', 'w')
f.close()
```

অথবা with ব্যবহার করা যেতে পারে:

```
with open('empty.txt', 'w') as f:
    pass
```

File-এ লেখা বা **(writing in a file)**

```
with open('test.txt', 'w', encoding='utf-8') as f:
    f.write('Python File Handling Basics')
```

আউটপুট:

(test.txt ফাইলে “Python File Handling Basics” লেখা হবে)

Append mode ব্যবহার করলে পুরনো টেক্সটের শেষে নতুন টেক্সট যুক্ত হয়।

```
with open('test.txt', 'a', encoding='utf-8') as f:
    f.write('\nনতুন লাইন যোগ করা হলো')
```

File পড়া বা **(Reading from a File)**

ফাইল পড়ার জন্য 'r' মোড ব্যবহার করা হয়।

```
with open('test.txt', 'r', encoding='utf-8') as f:
    data = f.read()
print(data)
```

Line by Line পড়া

```
with open('story.txt', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())
```

File বন্ধ করা বা (Closing a File)

ফাইল কাজ শেষ হলে অবশ্যই বন্ধ করতে হয়, নাহলে data loss হতে পারে।

```
f = open('data.txt', 'r')
# কাজ শেষ হলে
f.close()
```

তবে with statement ব্যবহার করলে এটি স্বয়ংক্রিয়ভাবে close হয়ে যায়।

```
with open('data.txt', 'r') as f:
    content = f.read()
```

Common Errors

Error	কারণ
FileNotFoundError	ফাইল না পাওয়া গেলে
PermissionError	ফাইলে write করার অনুমতি না থাকলে
UnicodeDecodeError	ভুল encoding ব্যবহার করলে

উদাহরণ:

```
try:
    with open('missing.txt', 'r', encoding='utf-8') as f:
        print(f.read())
except FileNotFoundError:
    print('ফাইল পাওয়া যায়নি!')
```

File Path (Absolute এবং Relative)

Absolute Path: যেমন C:\Users\User\data.txt
Relative Path: যেমন ./data.txt বা ../folder/file.txt
pathlib module ব্যবহার করলে path cross-platform হয়।

```
from pathlib import Path

p = Path('folder') / 'file.txt'
with p.open('w', encoding='utf-8') as f:
```

```
    f.write('Pathlib দিয়ে ফাইল তৈরি')
```

মূল পয়েন্টসমূহ

- File Handling data সংরক্ষণের জন্য ব্যবহৃত হয়।
- open() দিয়ে ফাইল খোলা হয় এবং mode নির্ধারণ করা হয়।
- ফাইল লেখার সময় 'utf-8' encoding ব্যবহার করা উচিত।
- with statement ফাইল auto-close করে।
- সাধারণত ব্যবহৃত mode: 'r', 'w', 'a', 'x', 'b', 't', '+'।

Practice Exercise

- data.txt নামে ফাইল তৈরি করে “Hello Bangladesh!” লিখো।
- একই ফাইলটি আবার read করে print করো।
- Append মোডে “Python Rocks!” যোগ করো।

উদাহরণ:

```
# Step 1
with open('data.txt', 'w', encoding='utf-8') as f:
    f.write('Hello Bangladesh!')

# Step 2
with open('data.txt', 'r', encoding='utf-8') as f:
    print(f.read())

# Step 3
with open('data.txt', 'a', encoding='utf-8') as f:
    f.write('\nPython Rocks!')
```

File Read

File Read কী

File read হলো একটি প্রক্রিয়া যার মাধ্যমে আমরা Python প্রোগ্রামে কোনো ফাইলের ভেতরের ডেটা পড়তে পারি। সাধারণত এটি open() function ব্যবহার করে 'r' (read) mode-এ করা হয়।

উদাহরণ:

```
f = open('sample.txt', 'r', encoding='utf-8')
data = f.read()
print(data)
f.close()
```

আউটপুট:

(sample.txt ফাইলে যা আছে তা টার্মিনালে প্রিন্ট হবে)

ফাইল পড়ার বিভিন্ন পদ্ধতি **(Different Ways to Read a File)**

Python ফাইল পড়ার জন্য কয়েকটি আলাদা method প্রদান করে।

1. read()

পুরো ফাইল একবারে পড়ে একটি string হিসেবে ফেরত দেয়।

```
with open('data.txt', 'r', encoding='utf-8') as f:
   text = f.read()
print(text)
```

2. readline()

প্রতিবার একটি করে লাইন পড়ে।

```
with open('story.txt', 'r', encoding='utf-8') as f:
   line1 = f.readline()
   line2 = f.readline()
print(line1)
print(line2)
```

3. readlines()

ফাইলের সব লাইনকে list আকারে ফেরত দেয়।

```
with open('story.txt', 'r', encoding='utf-8') as f:
   lines = f.readlines()
print(lines)
```

4. Loop ব্যবহার করে read করা

বড় ফাইল পড়ার জন্য সবচেয়ে উপযোগী পদ্ধতি।

```
with open('story.txt', 'r', encoding='utf-8') as f:
   for line in f:
      print(line.strip())
```

File Pointer or Cursor

প্রতিবার read করার পর file pointer ফাইলের শেষে চলে যায়। পুনরায় পড়ার আগে সেটিকে শুরুতে আনতে হয়।

```
f = open('data.txt', 'r', encoding='utf-8')
print(f.read(5))   # প্রথম ৫ অক্ষর পড়বে
print(f.read())    # বাকি অংশ পড়বে
f.seek(0)          # কার্সর আবার শুরুতে
print(f.read(10))  # আবার শুরু থেকে পড়া
```

```
f.close()
```

Read some specific parts

read(n) ব্যবহার করলে নির্দিষ্ট সংখ্যা অনুযায়ী character পড়া যায়।

```
with open('story.txt', 'r', encoding='utf-8') as f:
    part = f.read(20)
print(part)
```

Exception Handling সহ File Read

ফাইল না থাকলে বা path ভুল হলে error হতে পারে।

```
try:
    with open('missing.txt', 'r', encoding='utf-8') as f:
        print(f.read())
except FileNotFoundError:
    print('ফাইল পাওয়া যায়নি!')
```

Binary Mode-এ ফাইল পড়া

যখন ইমেজ বা পিডিএফ-এর মতো বাইনারি ডেটা পড়তে হয়, তখন 'rb' mode ব্যবহার করতে হয়।

```
with open('photo.jpg', 'rb') as f:
    data = f.read()
print(len(data), 'bytes পড়া হয়েছে')
```

মূল পয়েন্টসমূহ

- 'r' mode ফাইল পড়ার জন্য ব্যবহৃত হয়।
- read() পুরো ফাইল একসাথে পড়ে, বড় ফাইলের জন্য উপযুক্ত নয়।
- readline() প্রতি কল-এ এক লাইন পড়ে।
- readlines() সব লাইন list আকারে দেয়।
- Loop ব্যবহার করলে memory-efficientভাবে ফাইল পড়া যায়।
- Binary ফাইল পড়ার জন্য 'rb' mode ব্যবহার করতে হয়।

Practice Exercise

- story.txt ফাইল তৈরি করে তিনটি লাইন লিখো।
- read() দিয়ে পুরো কন্টেন্ট পড়ো।
- readline() দিয়ে একে একে লাইনগুলো পড়ো।
- Loop ব্যবহার করে প্রতিটি লাইন প্রিন্ট করো।

উদাহরণ:

```
# Step 1
with open('story.txt', 'w', encoding='utf-8') as f:
```

```
    f.write('Line 1\nLine 2\nLine 3')

# Step 2
with open('story.txt', 'r', encoding='utf-8') as f:
    print(f.read())

# Step 3
with open('story.txt', 'r', encoding='utf-8') as f:
    print(f.readline())
    print(f.readline())

# Step 4
with open('story.txt', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())
```

File Write and Append

File Write and Append কী?

File Write and Append হলো এমন একটি প্রক্রিয়া যার মাধ্যমে আমরা Python প্রোগ্রামে কোনো ফাইলে নতুন ডেটা write করা বা বিদ্যমান ফাইলে নতুন ডেটা append করা যায়। এটি open() function ব্যবহার করে 'w' (write) বা 'a' (append) mode-এ করা হয়।

উদাহরণ:

```
# Write mode ব্যবহার
f = open('sample.txt', 'w', encoding='utf-8')
f.write('This is new data.')
f.close()

# Append mode ব্যবহার
f = open('sample.txt', 'a', encoding='utf-8')
f.write('\nThis line is appended.')
f.close()
```

আউটপুট:

(sample.txt ফাইলে প্রথম লাইন লেখা হবে এবং দ্বিতীয়বার চালালে নতুন লাইন append হবে)

Different Ways to Write and Append a File

Python-এ ফাইল write করা বা append করার জন্য কয়েকটি পদ্ধতি রয়েছে।

1. write()
একটি string ফাইলে write করে।

```
with open('data.txt', 'w', encoding='utf-8') as f:
    f.write('Hello from write mode.')
```

2. writelines()
একাধিক string (list বা tuple) ফাইলে write করে।

```
lines = ['Line 1\n', 'Line 2\n', 'Line 3\n']
with open('multi.txt', 'w', encoding='utf-8') as f:
    f.writelines(lines)
```

3. Append Mode ('a') ব্যবহার করা
যদি ফাইলের শেষে নতুন ডেটা append করতে চান, 'a' mode ব্যবহার করতে হবে। এটি পুরনো ডেটা না মুছে নতুন কন্টেন্ট যোগ করে।

```
with open('log.txt', 'a', encoding='utf-8') as f:
    f.write('New log entry added.\n')
```

Overwrite বনাম **Append**

'w' mode ব্যবহারের সময় পুরনো ডেটা মুছে যায়, কিন্তু 'a' mode ব্যবহারে আগের কন্টেন্ট অপরিবর্তিত থাকে এবং নতুন লেখা শেষে যোগ হয়।

উদাহরণ:

```
# Overwrite
with open('demo.txt', 'w', encoding='utf-8') as f:
    f.write('First line only.')

# Append
with open('demo.txt', 'a', encoding='utf-8') as f:
    f.write('\nSecond line added.')
```

আউটপুট

```
First line only.
Second line added.
```

ফাইলে নতুন লাইন **Write** করা

নতুন লাইনে টেক্সট write করতে \n ব্যবহার করা হয়।

```
with open('tasks.txt', 'a', encoding='utf-8') as f:
```

```
    f.write('Task 1 completed\n')
    f.write('Task 2 completed\n')
```

Encoding এবং বাংলা **Write** করা

বাংলা বা Unicode টেক্সট write করার সময় অবশ্যই 'utf-8' encoding ব্যবহার করতে হয়।

```
with open('bangla.txt', 'w', encoding='utf-8') as f:
    f.write('হ্যালো পাইথন!\nবাংলা লেখা এখন খুব সহজ।')
```

Exception Handling সহ **File Write**

যদি ফাইলের path ভুল হয় বা permission না থাকে, তাহলে error উঠতে পারে।

```
try:
    with open('restricted/data.txt', 'w', encoding='utf-8') as f:
        f.write('Testing permission error.')
except PermissionError:
    print('ফাইল write করার অনুমতি নেই!')
```

Binary Mode-এ File Write করা

Binary ডেটা যেমন image বা pdf write করার সময় 'wb' mode ব্যবহার করা হয়।

```
with open('image_copy.jpg', 'wb') as f:
    f.write(b'\xff\xd8\xff\xe0')
```

মূল পয়েন্টসমূহ

- 'w' mode ফাইল নতুন করে তৈরি করে এবং পুরনো কন্টেন্ট মুছে দেয়।
- 'a' mode পুরনো কন্টেন্ট রেখে শেষে নতুন ডেটা append করে।
- write() string write করার জন্য ব্যবহৃত হয়।
- writelines() list বা tuple write করার জন্য ব্যবহৃত হয়।
- 'utf-8' encoding বাংলা write করার জন্য জরুরি।
- Binary ফাইলের জন্য 'wb' mode ব্যবহার করতে হয়।

Practice Exercise

- notes.txt নামে ফাইল তৈরি করে “First line from write mode.” write করো।
- 'a' mode ব্যবহার করে “Second line appended.” append করো।
- writelines() ব্যবহার করে আরও কয়েকটি লাইন write করো।
- ফাইল খুলে কন্টেন্ট read করো।

উদাহরণ:

```
# Step 1
with open('notes.txt', 'w', encoding='utf-8') as f:
    f.write('First line from write mode.')
```

```
# Step 2
with open('notes.txt', 'a', encoding='utf-8') as f:
    f.write('\nSecond line appended.')

# Step 3
more_lines = ['\nLine 3', '\nLine 4', '\nLine 5']
with open('notes.txt', 'a', encoding='utf-8') as f:
    f.writelines(more_lines)

# Step 4
with open('notes.txt', 'r', encoding='utf-8') as f:
    print(f.read())
```

File Pointer

File Pointer কী

File Pointer (বা File Cursor) হলো ফাইলের ভেতরে Python যে অবস্থান থেকে read করা বা write করা শুরু করবে, সেটিকে নির্দেশ করে। যখনই ফাইল খোলা হয়, কার্সর শুরুতে (position 0) থাকে এবং প্রতিবার read বা write করার পর তা সামনে এগিয়ে যায়।

উদাহরণ:

```
f = open('data.txt', 'r', encoding='utf-8')
content = f.read(5)
print('প্রথম ৫ অক্ষর:', content)
position = f.tell()
print('বর্তমান cursor position:', position)
f.close()
```

আউটপুট:

```
প্রথম ৫ অক্ষর: Hello
বর্তমান cursor position: 5
```

File Pointer Movement (tell() এবং **seek())**

Python-এ দুটি গুরুত্বপূর্ণ function আছে যেগুলোর মাধ্যমে File Pointer নিয়ন্ত্রণ করা হয় —

1. tell()

tell() function কার্সরের বর্তমান অবস্থান (byte সংখ্যা) জানায়।

```
with open('info.txt', 'r', encoding='utf-8') as f:
    f.read(10)
    print('Cursor position:', f.tell())
```

2. seek(offset, whence)
seek() ব্যবহার করে কার্সরকে নির্দিষ্ট অবস্থানে সরানো যায়।

offset → কয় byte সরবে
whence → কোন দিক থেকে সরবে (default = 0)
whence মান অর্থ

0	শুরু থেকে	ফাইলের শুরু থেকে গণনা
1	বর্তমান অবস্থান	বর্তমান position থেকে
2	শেষ থেকে	ফাইলের শেষ থেকে

উদাহরণ:

```
with open('example.txt', 'r', encoding='utf-8') as f:
    print('Start:', f.read(6))
    f.seek(0)
    print('Back to start:', f.read(6))
```

আউটপুট:

```
Start: Python
Back to start: Python
```

Example of Cursor Movement

```
with open('story.txt', 'r', encoding='utf-8') as f:
    print(f.read(4))       # প্রথম ৪ অক্ষর পড়া
    print('Position:', f.tell())
    f.seek(0)              # Cursor আবার শুরুতে
    print(f.read(8))       # আবার প্রথম ৮ অক্ষর পড়া
```

আউটপুট:

```
PyTh
Position: 4
PyThon I
```

File Pointer in Write Mode

Write mode-এ ফাইল খুললে pointer শুরুতে থাকে এবং পুরনো কন্টেন্ট truncate হয়ে যায়। Append mode-এ কার্সর সবসময় ফাইলের শেষে থাকে।

উদাহরণ:

```
# Write Mode
a = open('sample.txt', 'w', encoding='utf-8')
```

```
a.write('ABC')
print(a.tell())  # position 3

a.close()

# Append Mode
b = open('sample.txt', 'a', encoding='utf-8')
print(b.tell())  # ফাইলের শেষে
b.write('\nDEF')
b.close()
```

Random Access Using seek()

seek() function ব্যবহার করে ফাইলের যেকোনো অংশে সরাসরি গিয়ে data read করা বা write করা যায়।

```
with open('data.txt', 'r', encoding='utf-8') as f:
    f.seek(7)
    part = f.read(5)
    print('Selected part:', part)
```

Binary Mode Pointer Example

Binary ফাইলেও pointer কাজ করে, তবে এটি byte লেভেলে।

```
with open('image.jpg', 'rb') as f:
    f.seek(100)
    data = f.read(20)
    print('Read bytes:', len(data))
```

মূল পয়েন্টসমূহ

- File Pointer নির্দেশ করে ফাইলের কোন অবস্থানে কাজ হচ্ছে।
- tell() pointer-এর বর্তমান অবস্থান জানায়।
- seek() pointer-কে নির্দিষ্ট অবস্থানে নিয়ে যায়।
- 'w' mode-এ pointer শুরু থেকে থাকে এবং পুরনো data মুছে ফেলে।
- 'a' mode-এ pointer সবসময় শেষে থাকে।
- Binary mode-এ pointer byte লেভেলে কাজ করে।

Practice Exercise

- data.txt ফাইল তৈরি করে কিছু টেক্সট write করো।
- tell() ব্যবহার করে কার্সরের অবস্থান বের করো।
- seek() ব্যবহার করে শুরুতে ফিরে গিয়ে প্রথম ৫ অক্ষর read করো।
- ফাইলের মাঝখান থেকে নির্দিষ্ট অংশ read করো।

উদাহরণ:

```
# Step 1
with open('data.txt', 'w', encoding='utf-8') as f:
    f.write('Python File Pointer Example')

# Step 2
with open('data.txt', 'r', encoding='utf-8') as f:
    f.read(10)
    print('Position:', f.tell())

# Step 3
    f.seek(0)
    print('First 5:', f.read(5))

# Step 4
    f.seek(7)
    print('Middle part:', f.read(6))
```

Practice Problem on File

Practice Problem 1: Write and Read from File

একটি প্রোগ্রাম লিখো যা একটি টেক্সট ফাইল তৈরি করে তাতে কিছু টেক্সট write করে, এরপর সেই ফাইল খুলে আবার read করে।

উদাহরণ:

```
# Writing to file
with open('practice1.txt', 'w', encoding='utf-8') as f:
    f.write('This is a practice file.\nLearning File Handling in Python.')

# Reading from file
with open('practice1.txt', 'r', encoding='utf-8') as f:
    content = f.read()
    print(content)
```

আউটপুট:

```
This is a practice file.
Learning File Handling in Python.
```

Practice Problem 2: Count Lines, Words, and Characters

একটি ফাইল থেকে মোট লাইন, শব্দ এবং অক্ষর গণনা করো।

```
with open('practice2.txt', 'r', encoding='utf-8') as f:
```

```
    lines = f.readlines()
    line_count = len(lines)
    word_count = sum(len(line.split()) for line in lines)
    char_count = sum(len(line) for line in lines)

print('Lines:', line_count)
print('Words:', word_count)
print('Characters:', char_count)
```
আউটপুট:

```
Lines: 3
Words: 10
Characters: 52
```

Practice Problem 3: Copy File Content
একটি প্রোগ্রাম লিখো যা একটি ফাইলের কন্টেন্ট অন্য ফাইলে কপি করে।

```
with open('source.txt', 'r', encoding='utf-8') as src:
    data = src.read()

with open('destination.txt', 'w', encoding='utf-8') as dest:
    dest.write(data)

print('File copied successfully!')
```

Practice Problem 4: Append Data to an Existing File
একটি বিদ্যমান ফাইলে নতুন লাইন append করো এবং তারপর পুরো ফাইলটি read করো।

```
with open('log.txt', 'a', encoding='utf-8') as f:
    f.write('\nNew log entry added.')

with open('log.txt', 'r', encoding='utf-8') as f:
    print(f.read())
```

Practice Problem 5: Search for a Word in File
একটি প্রোগ্রাম লিখো যা ফাইলের মধ্যে কোনো নির্দিষ্ট শব্দ আছে কিনা তা যাচাই করবে।

```
word_to_find = 'Python'
found = False

with open('text.txt', 'r', encoding='utf-8') as f:
    for line in f:
        if word_to_find in line:
            found = True
```

```
            break

if found:
    print(f'Word "{word_to_find}" found in file!')
else:
    print(f'Word "{word_to_find}" not found!')
```

Practice Problem 6: Reverse Content of a File

একটি ফাইলের কন্টেন্ট উল্টে (reverse করে) অন্য ফাইলে সংরক্ষণ করো।

```
with open('data.txt', 'r', encoding='utf-8') as f:
    content = f.read()

with open('reverse.txt', 'w', encoding='utf-8') as f:
    f.write(content[::-1])
```

Practice Problem 7: Read File Using Loop

একটি বড় ফাইলকে line by line read করে প্রতিটি লাইনের আগে লাইন নম্বর দেখাও।

```
with open('story.txt', 'r', encoding='utf-8') as f:
    for i, line in enumerate(f, start=1):
        print(f'{i}: {line.strip()}')
```

Exceptions and Why Handling Them

What is an Exception

Exception হলো প্রোগ্রাম চলার সময় ঘটে যাওয়া runtime error, যা প্রোগ্রামের স্বাভাবিক প্রবাহ বন্ধ করে দেয়। উদাহরণস্বরূপ, যদি কোনো ফাইল না থাকে এবং আমরা সেটি খুলতে চাই, তাহলে Python একটি FileNotFoundError exception দেয়।

উদাহরণ:

```
f = open('nofile.txt', 'r', encoding='utf-8')
print(f.read())
```

আউটপুট:

```
FileNotFoundError: [Errno 2] No such file or directory: 'nofile.txt'
```

Why Handle Exceptions

Exception handle না করলে প্রোগ্রাম crash করে যায়। Exception handling-এর মাধ্যমে প্রোগ্রামকে আরও robust ও user-friendly করা যায়।

উদাহরণ:

```
try:
    f = open('nofile.txt', 'r', encoding='utf-8')
    data = f.read()
    print(data)
except FileNotFoundError:
    print('File not found! Please check the file name.')
```

আউটপুট:

```
File not found! Please check the file name.
```

এখানে error ঘটলেও প্রোগ্রাম বন্ধ হয়নি।

Common Exceptions in File Handling

Exception	Cause
FileNotFoundError	ফাইল না পাওয়া গেলে
PermissionError	ফাইলে পড়া/লেখার অনুমতি না থাকলে
IsADirectoryError	ফাইলের জায়গায় ডিরেক্টরি ব্যবহার করলে
UnicodeDecodeError	ভুল encoding ব্যবহার করলে

Using try-except Block

Python-এ exception handle করার জন্য try-except structure ব্যবহৃত হয়।

```
try:
    f = open('data.txt', 'r', encoding='utf-8')
    print(f.read())
except FileNotFoundError:
    print('File does not exist!')
except PermissionError:
    print('You do not have permission to read this file.')
```

Using else Block

else block তখনই চলে যখন try অংশে কোনো exception না ঘটে।

```
try:
    f = open('data.txt', 'r', encoding='utf-8')
except FileNotFoundError:
    print('File not found!')
```

```
else:
    print('File opened successfully!')
    print(f.read())
    f.close()
```

Using finally Block

finally block সবসময় execute হয়, error হোক বা না হোক। সাধারণত resource (যেমন file বা database) বন্ধ করার জন্য এটি ব্যবহৃত হয়।

```
try:
    f = open('data.txt', 'r', encoding='utf-8')
    print(f.read())
except FileNotFoundError:
    print('File not found!')
finally:
    print('File handling process finished.')
```

আউটপুট:

```
File not found!
File handling process finished.
```

Multiple Exceptions

একাধিক exception একসাথে ধরতে tuple ব্যবহার করা যায়।

```
try:
    f = open('file.txt', 'r', encoding='utf-8')
    print(f.read())
except (FileNotFoundError, PermissionError):
    print('File access problem!')
```

Raising Custom Exceptions

নিজস্ব error message তৈরি করার জন্য raise keyword ব্যবহার করা হয়।

```
filename = ''
if not filename:
    raise ValueError('Filename cannot be empty!')
```

আউটপুট:

```
ValueError: Filename cannot be empty!
```

মূল পয়েন্টসমূহ

- Exception হলো runtime error, যা প্রোগ্রামকে বন্ধ করে দিতে পারে।
- try-except block ব্যবহার করে প্রোগ্রাম crash হওয়া ঠেকানো যায়।
- else block তখনই চলে যখন কোনো error না ঘটে।

- finally block সবসময় execute হয়।
- raise দিয়ে custom exception তোলা যায়।

Practice Exercise
- এমন একটি প্রোগ্রাম লিখো যা ফাইল খুলবে, যদি ফাইল না থাকে তাহলে FileNotFoundError handle করবে।
- ফাইল পড়া শেষে finally ব্লকে “Program ended” প্রিন্ট করো।
- ফাইলের নাম খালি থাকলে ValueError তোল।

উদাহরণ**:**

```
try:
    filename = input('Enter file name: ')
    if not filename:
        raise ValueError('Filename cannot be empty!')

    with open(filename, 'r', encoding='utf-8') as f:
        print(f.read())

except FileNotFoundError:
    print('File not found!')
except ValueError as e:
    print(e)
finally:
    print('Program ended.')
```

সারসংক্ষেপ
Exception handling প্রোগ্রামকে আরও নিরাপদ ও error-tolerant করে তোলে।
try-except-else-finally structure ব্যবহার করে যেকোনো error সুন্দরভাবে handle করা যায়।

Try, Except, Else and Finally

What these blocks do

try, except, else, এবং finally ব্যবহার করে Python প্রোগ্রামে exception handling করা হয়। লক্ষ্য হলো error ঘটলে প্রোগ্রাম যেন gracefully behave করে এবং resource (যেমন file) সবসময় সঠিকভাবে বন্ধ হয়।

সংক্ষিপ্ত ব্যাখ্যা:

- try: এমন কোড রাখি যেখানে exception ঘটতে পারে।
- except: নির্দিষ্ট/সাধারণ exception ধরি এবং তার জন্য হ্যান্ডলিং লিখি।
- else: যদি try অংশে কোনো exception না ঘটে, তখন চলে।

- finally: exception হোক বা না হোক—সবসময় চলে; cleanup-এর জন্য ভালো।

Basic Syntax

```
try:
   risky_operation()
except SomeError:
   handle_error()
else:
   run_if_no_error()
finally:
   clean_up_always()
```

Control Flow (Text Diagram)

try চালাও →
যদি exception ঘটে → মেলানো except ব্লকে যাও → finally → সমাপ্ত।
যদি exception না ঘটে → else চালাও → finally → সমাপ্ত।

File Handling Example

```
try:
   f = open('data.txt', 'r', encoding='utf-8')  # read করা
   text = f.read()
except FileNotFoundError:
   print('File not found!')
else:
   print('Length:', len(text))
finally:
   try:
      f.close()
   except NameError:
      pass  # f তৈরি হয়নি
```

ব্যাখ্যা: ফাইল না পেলে except চলে; পেলে else-এ কাজ হয়; শেষে finally দিয়ে close করার চেষ্টা করা হয়।

Using with-statement vs finally

with open(...) নিজেই auto-close করে; অনেক ক্ষেত্রে finally লিখতে হয় না।

```
try:
   with open('notes.txt', 'r', encoding='utf-8') as f:
      print(f.readline())
except FileNotFoundError:
   print('Missing file')
```

Multiple except Blocks

```
try:
    with open('conf.txt', 'r', encoding='utf-8') as f:
        value = int(f.read().strip())
except FileNotFoundError:
    print('Configuration file missing')
except ValueError:
    print('Configuration must be an integer')
```

Catching Multiple Exceptions Together

```
try:
    connect_and_fetch()
except (TimeoutError, ConnectionError) as e:
    print('Network issue:', e)
```

Generic except and Exception Object

শুধু except: সব error ধরে, তবে best practice হলো নির্দিষ্ট exception ধরা।

```
try:
    do_work()
except Exception as e:  # generic, সতর্কতার সাথে ব্যবহার করুন
    print('Unexpected error:', type(e).__name__, e)
```

else Block: When to Use

try সফল হলে যে post-processing দরকার, সেটি else-এ রাখলে intent পরিষ্কার হয় এবং exception handling logic আলাদা থাকে।

```
try:
    n = int(input('Enter number: '))
except ValueError:
    print('Please enter a valid integer')
else:
    print('Square =', n*n)
```

finally Block: Cleanup

ফাইল/নেটওয়ার্ক/ডাটাবেস close করা, temporary resource delete করা, lock release করা—এসবের জন্য finally নির্ভরযোগ্য।

```
lock.acquire()
try:
    critical_section()
finally:
    lock.release()
```

Best Practices

- নির্দিষ্ট exception ধরুন; অপ্রয়োজনে generic except Exception ব্যবহার করবেন না।
- Exception swallow করবেন না; অন্তত অর্থপূর্ণ message/log দিন।
- Resource management-এর জন্য with প্যাটার্ন ব্যবহার করুন।
- else ব্লক ব্যবহার করে success-path আলাদা রাখুন।
- finally শুধু cleanup-এর জন্য রাখুন; সেখানে এমন কোড দেবেন না যাতে নতুন exception ওঠে (অথবা সেগুলোও handle করুন)।

Common Mistakes

- খালি except: ব্যবহার করে সব error গিলে ফেলা।
- finally-তে এমন কাজ করা যা আবার নতুন exception তোলে কিন্তু ধরা হয় না।
- try ব্লকে অতিরিক্ত বড় কোড রাখা—scope ছোট রাখুন যাতে কোন লাইনে error হতে পারে বোঝা যায়।

Practice Exercises

- User থেকে একটি filename ইনপুট নাও। ফাইলটি read করা সম্ভব হলে প্রথম লাইন প্রিন্ট করো; না হলে অর্থপূর্ণ error দেখাও। else এবং finally ব্যবহার করো।
- দুইটি সংখ্যা ইনপুট নিয়ে ভাগফল বের করো। ValueError (invalid input) এবং ZeroDivisionError আলাদা except ব্লকে handle করো।
- একটি temporary file তৈরি করে তাতে ডেটা write করা এবং শেষে delete করা—try/except/else/finally দিয়ে সম্পন্ন করো।

রেফারেন্স সমাধান **(Exercise 2):**

```
try:
    a = float(input('A: '))
    b = float(input('B: '))
    result = a / b
except ValueError:
    print('Enter valid numbers')
except ZeroDivisionError:
    print('Division by zero is not allowed')
else:
    print('Result =', result)
finally:
    print('Done')
```

OOP in Python

Introduction to OOP

প্রোগ্রামিং শেখার প্রাথমিক ধাপেই আমরা বিভিন্ন ডেটা টাইপের সঙ্গে পরিচিত হই — যেমন string, list, dictionary, tuple ইত্যাদি। এই ডেটা টাইপগুলো আমাদের বিভিন্ন ধরনের তথ্য সংরক্ষণ ও প্রক্রিয়াকরণের সুবিধা দেয়। কিন্তু সময়ের সাথে প্রোগ্রামের জটিলতা বাড়তে থাকলে শুধু এই বেসিক ডেটা টাইপ দিয়ে সবকিছু সামলানো কঠিন হয়ে পড়ে।

কেন **OOP** প্রয়োজন**?**

ধরুন, আপনি একটি মেশিন লার্নিং প্রজেক্ট করছেন যেখানে ছাত্রছাত্রীদের তথ্য, তাদের স্কোর, এবং মডেল ট্রেইনিং ডেটা নিয়ে কাজ করতে হচ্ছে। যদি সব তথ্য আলাদা আলাদা list, dict, বা variable-এ রাখেন, তাহলে কোডটি খুব দ্রুত জটিল হয়ে যাবে।

উদাহরণস্বরূপ:

```
student_names = ["রিমন", "সাবিনা", "অনিক"]
student_marks = [85, 90, 78]
student_ids = [101, 102, 103]
```

এভাবে ডেটা রাখা গেলেও, যদি কোনো ছাত্রের সমস্ত তথ্য একসাথে দরকার হয়, তখন একাধিক তালিকা থেকে সেই তথ্য সংগ্রহ করতে হবে। এটা শুধু সময়সাপেক্ষ নয়, বরং কোডের ভুল হওয়ার সম্ভাবনাও বাড়ে।

এখানেই Object-Oriented Programming (OOP) আমাদের সাহায্য করে।

OOP ধারণা অনুযায়ী, আমরা ডেটা ও ডেটার ওপর কাজ করা ফাংশনগুলোকে একসাথে একটি “অবজেক্ট”-এর মধ্যে বাঁধতে পারি। ফলে প্রোগ্রামটি হয় আরো সংগঠিত, সহজবোধ্য, এবং পুনঃব্যবহারযোগ্য (reusable)।

মেশিন লার্নিং **(ML)-**এর সাথে **OOP-**এর সম্পর্ক

মেশিন লার্নিং শেখার সময় আমরা মূলত বিভিন্ন মডেল, ডেটা প্রসেসিং পাইপলাইন, ও ট্রেনিং প্রক্রিয়া নিয়ে কাজ করি। এই প্রতিটি উপাদানকে OOP-এর ধারণা ব্যবহার করে আলাদা class ও object হিসেবে তৈরি করা যায়।

উদাহরণস্বরূপ:

Dataset class ডেটা লোড ও প্রিপ্রসেস করে
Model class মডেল ট্রেইন ও প্রেডিক্ট করে
Trainer class পুরো ট্রেইনিং লুপ পরিচালনা করে

এভাবে প্রজেক্টের প্রতিটি অংশ আলাদা হলেও তারা পরস্পরের সাথে সুন্দরভাবে যুক্ত থাকে।

Class and Objects Theory

প্রথম মডিউলে আমরা আলোচনা করেছি কেন OOP প্রয়োজন, এবং কীভাবে এটি কোডকে আরও সংগঠিত ও পুনঃব্যবহারযোগ্য করে তোলে।

এবার আমরা OOP-এর দুইটি মূল স্তম্ভের দিকে আসব — Class এবং Object।

এই দুই ধারণাই OOP-এর ভিত্তি। একটিকে না বুঝলে অন্যটি কখনো পূর্ণভাবে বোঝা যায় না।

ক্লাস **(Class)** কী**?**

সহজভাবে বলতে গেলে, ক্লাস হলো কোনো কিছুর নকশা বা ছক (Blueprint)।

এটা এমন একটা ছাঁচ, যার উপর ভিত্তি করে আমরা বাস্তব জগতের জিনিসগুলোকে প্রোগ্রামে তৈরি করি।

উদাহরণস্বরূপ, ধরুন আমরা একটি Phone নিয়ে ভাবছি।

যে কোনো ফোনের কিছু সাধারণ বৈশিষ্ট্য থাকে — যেমন:

ব্যাটারি ক্যাপাসিটি
প্রসেসর টাইপ
র‍্যাম ও স্টোরেজ
ক্যামেরা স্পেসিফিকেশন
এগুলো হলো ফোনের গুণাবলি (Attributes)।

তাছাড়া ফোনের কিছু কাজও থাকে, যেমন:

ফোন চালু করা বা বন্ধ করা
চার্জ দেওয়া
কল করা বা মেসেজ পাঠানো
এগুলো হলো ফোনের কার্যক্ষমতা বা আচরণ (Behaviors/Methods)।

এখন, “Phone” নামের একটি ক্লাস তৈরি করলে সেটি এই সমস্ত বৈশিষ্ট্য ও আচরণের একটি ধারণাগত কাঠামো হিসেবে কাজ করবে।

অর্থাৎ, ক্লাস কেবল বলে দেয় — “একটি ফোনে কী কী থাকা উচিত, এবং সে কী করতে পারবে।”

কিন্তু ক্লাস নিজে কোনো বাস্তব ফোন নয়; এটি শুধু তার ডিজাইন।

অবজেক্ট **(Object)** কী?

অবজেক্ট হলো সেই ক্লাসের বাস্তব রূপ (Instance)।

যখন আমরা “Phone” ক্লাস থেকে একটি আসল ফোন তৈরি করি — যেমন “Samsung Galaxy S24” বা “iPhone 15” — তখন সেটি একটি অবজেক্ট।

অর্থাৎ,

Class = নকশা (Design)
Object = বাস্তব রূপ (Real Entity)
প্রত্যেকটি অবজেক্ট একই ক্লাসের নকশা অনুসরণ করে তৈরি হয়, কিন্তু তাদের মান (value) ভিন্ন হতে পারে।

যেমন,

Samsung ফোনে ব্যাটারি 5000mAh
iPhone-এ ব্যাটারি 3500mAh
দু'টোই “Phone” ক্লাস থেকে এসেছে, কিন্তু তাদের নির্দিষ্ট বৈশিষ্ট্য আলাদা।

ক্লাস ও অবজেক্টের সম্পর্ক

ক্লাস এবং অবজেক্টের মধ্যে সম্পর্কটা অনেকটা “ব্লুপ্রিন্ট ও বিল্ডিং”-এর মতো।

ব্লুপ্রিন্ট ছাড়া বিল্ডিং বানানো যায় না (অর্থাৎ, ক্লাস ছাড়া অবজেক্ট হয় না)।
আবার ব্লুপ্রিন্ট থাকলেই বিল্ডিং তৈরি হয়ে যায় না (অর্থাৎ, ক্লাস লিখলেই প্রোগ্রামে কাজ শুরু হয় না; অবজেক্ট তৈরি করতে হয়)।
তাই,

ক্লাস হলো ধারণা (concept)।

অবজেক্ট হলো বাস্তব রূপ (instance)।

Class and Object implementation

আগের মডিউলে আমরা শিখেছিলাম, ক্লাস হচ্ছে কোনো কিছুর নকশা (Blueprint) এবং অবজেক্ট সেই নকশা অনুসারে তৈরি বাস্তব বস্তু (Instance)।

এবার আমরা দেখব, কিভাবে সেই ধারণাটি Python কোডের মাধ্যমে বাস্তবায়ন করা যায়।

ক্লাস: Phone

চলুন আমরা একটি ফোন কোম্পানির উদাহরণ নিই।

আমাদের প্রতিটি ফোনের কিছু সাধারণ গুণাবলি থাকবে — যেমন ব্যাটারি, ক্যামেরা, মডেল ইত্যাদি।

এছাড়াও ফোন কিছু কাজ করতে পারবে — যেমন চার্জ নেওয়া বা ছবি তোলা।

```
class Phone:
    category = "Electronics"     # Class Attribute

    # Constructor
    def __init__(self, model, battery, camera, battery_percentage=100):
        self.model = model                    # Instance Attribute
        self.battery = battery                # Instance Attribute
        self.camera = camera                  # Instance Attribute
        self.battery_percentage = battery_percentage  # Instance Attribute

    # Methods
    def charge(self, hour):
        print(f"charge completed by {hour}")

    def capture(self, photo):
        if self.battery_percentage <= 0:
            print("No charge")
        else:
            self.battery_percentage -= photo
            print(f"photo captured in {self.model}")
```

ক্লাস অ্যাট্রিবিউট **(Class Attribute)**

```
category = "Electronics"
```

এটি একটি ক্লাস অ্যাট্রিবিউট — অর্থাৎ এটি পুরো ক্লাসের জন্য একবারেই নির্ধারিত হয়।
সমস্ত অবজেক্ট (যেমন apple, blueberry, motorola) এই একই মানটি ভাগাভাগি করে ব্যবহার করে।

কোনো একটি অবজেক্টের জন্য এই মান পরিবর্তন করলে সেটি শুধুমাত্র ঐ অবজেক্টে প্রযোজ্য হয় না, বরং ক্লাসের উপর নির্ভরশীল থাকে।

যে কোনো ফোনই হোক না কেন — Samsung, Apple, Motorola — সবগুলোই “Electronics” শ্রেণির অন্তর্ভুক্ত।

তাই এটি ক্লাসেরই একটি বৈশিষ্ট্য, আলাদা আলাদা ফোনের নয়।

ইনস্ট্যান্স অ্যাট্রিবিউট **(Instance Attribute)**

```
def __init__(self, model, battery, camera, battery_percentage=100):
    self.model = model
    self.battery = battery
    self.camera = camera
    self.battery_percentage = battery_percentage
```

__init__() হলো constructor method — যা অবজেক্ট তৈরি হওয়ার সময় স্বয়ংক্রিয়ভাবে চালু হয়।
এখানে self প্রতিটি অবজেক্টের নিজস্ব (unique) রেফারেন্স বোঝায়।
self.model, self.battery, self.camera ইত্যাদি হলো ইনস্ট্যান্স অ্যাট্রিবিউট — অর্থাৎ প্রতিটি অবজেক্টের জন্য আলাদা মান সংরক্ষণ করে।

Apple ফোনের ব্যাটারি 3000mAh হতে পারে, কিন্তু Motorola-র ব্যাটারি 3500mAh — তাই এই মানগুলো অবজেক্টভেদে আলাদা থাকে।

মেথড **(Methods)**

charge() মেথড

```
def charge(self, hour):
    print(f"charge completed by {hour}")
```

এটি ফোনের একটি action বা আচরণ বোঝায়।

যখন ফোন চার্জ নেয়, তখন এটি একটি বার্তা প্রিন্ট করে।

এই মেথড ফোনের কাজের মধ্যে পড়ে।

capture() মেথড

```
def capture(self, photo):
    if self.battery_percentage <= 0:
        print("No charge")
    else:
        self.battery_percentage -= photo
        print(f"photo captured in {self.model}")
```

এই মেথডের মাধ্যমে ফোন ছবি তোলে।

প্রতিবার ছবি তোলার সময় ব্যাটারির কিছু শতাংশ কমে যায়।

যদি ব্যাটারি শেষ হয়ে যায়, তাহলে ফোন “No charge” বার্তা দেয়।

self.battery_percentage প্রতি অবজেক্টে আলাদা মানে পরিবর্তিত হয়।

অবজেক্ট তৈরি করা **(Creating Objects)**

```
apple = Phone("Iphone17", 3000, 40)
blueberry = Phone("B-17", 4000, 30)
motorola = Phone("M-17", 3500, 35)
```

এখন আমরা “Phone” ক্লাস থেকে তিনটি অবজেক্ট তৈরি করলাম।

প্রতিটি অবজেক্টের নিজস্ব model, battery, এবং camera আছে।

ধাপ ৫: মেথড কল করা ও মান দেখা

```
apple.capture(10)
print(motorola.battery_percentage)
```

apple.capture(10) মানে হলো Apple ফোন 10% ব্যাটারি ব্যবহার করে একটি ছবি তুলেছে।
motorola.battery_percentage দেখাবে Motorola ফোনের বর্তমান ব্যাটারি মান (এখনও 100%, কারণ এটি কোনো ছবি তোলে নি)।

OOP Inheritance

ইনহেরিটেন্স কী?

Inheritance শব্দের মানে হলো “উত্তরাধিকার সূত্রে পাওয়া।”
OOP–এ ইনহেরিটেন্স মানে হলো — একটি ক্লাস অন্য একটি ক্লাসের বৈশিষ্ট্য **(attributes)** ও ফাংশনালিটি **(methods)** উত্তরাধিকার সূত্রে পায়।
অর্থাৎ, আমরা আগের কোনো ক্লাসের কোড পুনরায় না লিখে, সেটি থেকে নতুন একটি ক্লাস তৈরি করতে পারি।
এটি কোডকে পরিষ্কার **(clean)**, সংগঠিত **(organized)**, এবং পুনঃব্যবহারযোগ্য **(reusable)** করে।

কেন ইনহেরিটেন্স দরকার?

আমরা ইতিমধ্যেই একটি Phone ক্লাস লিখেছি, যেখানে ব্যাটারি, ক্যামেরা, এবং চার্জ দেওয়ার ফিচার আছে।
এখন আমরা “SmartPhone” নামে নতুন একটি ক্লাস বানাতে চাই — যেখানে আগের সব ফিচার থাকবে, তবে কিছু নতুন ক্ষমতাও যোগ হবে, যেমন প্রসেসর এবং ফাস্ট চার্জিং।
এক্ষেত্রে আমাদের Phone ক্লাসের কোড আবার লেখার দরকার নেই।
আমরা চাইলে সেটি ইনহেরিট করতে পারি।

Single Inheritance

একক ইনহেরিটেন্সে একটি **child class (derived class)** একটি **parent class (base class)**–এর বৈশিষ্ট্য ও মেথড পায়।
চলুন দেখি উদাহরণটি:

```
# Parent class

class Phone:

    category = "Electronics"



    def __init__(self, model, battery, camera, battery_percentage=100):

        self.model = model

        self.battery = battery
```

```
        self.camera = camera
        self.battery_percentage = battery_percentage


    def charge(self, hour):
        print(f"charge completed by {hour}")


    def capture(self, photo):
        if self.battery_percentage <= 0:
            print("No charge")
        else:
            self.battery_percentage -= photo
            print(f"photo captured in {self.model}")
```

এখন আমরা একটি **child class** তৈরি করব — SmartPhone, যা Phone ক্লাস থেকে ইনহেরিট করছে।

```
# Child / Derived class
class SmartPhone(Phone):
    def __init__(self, model, battery, camera, processor):
        super().__init__(model, battery, camera)   # parent class constructor call
        self.processor = processor                 # new attribute for SmartPhone

    # Modified method
    def charge(self, hour):
        print("fast charging in process")
        super().charge(hour)  # parent class-এর charge মেথডও ডাকা হচ্ছে
```

এখন আমরা একটি অবজেক্ট তৈরি করি:

```
pro = SmartPhone("X", 5000, 100, "SnapDragon")

print(pro.model)    # Phone ক্লাস থেকে পাওয়া
```

```
pro.charge(1)       # SmartPhone-এর modified মেথড
```

এখানে কী হচ্ছে:

- SmartPhone ক্লাস Phone–এর সব attributes ও methods ইনহেরিট করছে।
- আমরা চাইলে নতুন attribute (processor) যোগ করতে পারি।
- চাইলে পুরনো method (charge) পরিবর্তন বা override করতে পারি।
- super() ব্যবহার করে parent class–এর method–ও একসাথে ডাকা যায়।

এভাবেই **Single Inheritance** আমাদের কোড পুনরায় ব্যবহার ও উন্নয়ন করতে সাহায্য করে।

Multiple Inheritance

Python–এ একাধিক ক্লাস থেকেও ইনহেরিট করা যায়।
এটিকে বলা হয় **Multiple Inheritance**।
ধরুন, আমরা চাই ফোনে **Cooling Mechanism** নামের আরেকটি সিস্টেম যোগ করতে।
এই সিস্টেমের জন্য আমরা আলাদা একটি ক্লাস তৈরি করব:

```
class Cooling_Mechanism:
    def __init__(self, cooling_method):
        self.cooling_method = cooling_method

    def cooling_on(self):
        return "Cooling system activated"
```

এখন আমরা চাই, SmartPhone–এর ফিচার এবং Cooling_Mechanism–এর ফিচার একত্রে একটি নতুন ক্লাসে থাকুক। সেটি হবে SmartPhone_CoolingMode।

```
class SmartPhone_CoolingMode(SmartPhone, Cooling_Mechanism):
    def __init__(self, model, battery, camera, processor, cooling_method):
        SmartPhone.__init__(self, model, battery, camera, processor)
        Cooling_Mechanism.__init__(self, cooling_method)
```

এখন আমরা নতুন অবজেক্ট তৈরি করব:

```
pro_cooling = SmartPhone_CoolingMode("Y", 5000, 100, "SD", "Nitrogen")

print(pro_cooling.processor)        # SmartPhone class থেকে
```

```
print(pro_cooling.battery)          # Phone class থেকে
print(pro_cooling.cooling_method)   # Cooling_Mechanism class থেকে
print(pro_cooling.cooling_on())     # Cooling_Mechanism-এর method
print(pro_cooling.charge(1))        # SmartPhone-এর modified charge
```

এখানে কী ঘটছে:

- SmartPhone_CoolingMode ক্লাস দুটি parent ক্লাস থেকে ইনহেরিট করছে:
- SmartPhone
- Cooling_Mechanism
- তাই এটি উভয় ক্লাসের গুণাবলি ও মেথড পায়।
- এটি আমাদের দেখায় কিভাবে OOP–এ বহুমুখী বৈশিষ্ট্য একত্রে যুক্ত করা যায়।

OOP Polymorphism

Polymorphism কী?

Polymorphism শব্দটি এসেছে গ্রিক ভাষা থেকে — “Poly” মানে অনেক, আর “Morph” মানে রূপ।
অর্থাৎ, একই জিনিসের একাধিক রূপ।
প্রোগ্রামিং–এ Polymorphism মানে হলো —একই নামের মেথড বা ফাংশন বিভিন্ন ক্লাসে ভিন্নভাবে কাজ করতে পারে।
এভাবে আমরা একই ইন্টারফেস ব্যবহার করে বিভিন্ন অবজেক্টের ভিন্ন আচরণ পেতে পারি।

কেন **Polymorphism** দরকার?

ধরুন, আমাদের কাছে তিন ধরনের ক্যামেরা আছে:

- **Smart Phone**
- **DSLR**
- **Drone**

তিনটিই ছবি তুলতে পারে — কিন্তু প্রত্যেকে নিজস্ব পদ্ধতিতে।
এই তিনটি ডিভাইসের জন্য আমরা আলাদা ক্লাস তৈরি করতে পারি,
কিন্তু তাদের সবার মধ্যে একটি সাধারণ মেথড থাকবে — capture()।
এখন Polymorphism–এর মাধ্যমে আমরা সেই capture() মেথডকে প্রতিটি ক্লাসে ভিন্নভাবে ব্যবহার করতে পারব।

Types of Polymorphism

Python–এ মূলত দুই ধরনের Polymorphism দেখা যায়:

Compile-time Polymorphism (Method Overloading)	একই নামের একাধিক মেথড, যেগুলো প্যারামিটারের সংখ্যায় বা ধরনে আলাদা হয়।
Run-time Polymorphism (Method Overriding)	চাইল্ড ক্লাসে প্যারেন্ট ক্লাসের মেথড নতুনভাবে সংজ্ঞায়িত করা হয়।

Method Overloading

Python সরাসরি method overloading সাপোর্ট করে না (যেমন Java বা C++–এ করে)।
তবে আমরা এটি default arguments বা variable arguments (*args, **kwargs) ব্যবহার করে অনুকরণ করতে পারি।
ধরুন, আমরা একটি ক্যামেরার zoom() মেথড তৈরি করি:

```
class Camera:
    def zoom(self, level=1):
        print(f"Zoomed {level}x")
```

এখানে যদি আমরা zoom() মেথডে প্যারামিটার না দিই, তাহলে এটি 1x জুম করে,
আর যদি মান দিই, তাহলে সেটি অনুযায়ী কাজ করে।
এভাবে একই মেথড ভিন্নভাবে কাজ করছে — এটিই **method overloading-**এর ধারণা।

Method Overriding

এখন আসুন মূল উদাহরণে যাই — এখানে তিনটি ক্লাস রয়েছে: Smart_Phone, DSLR, এবং Drone,
যেগুলো Camera ক্লাস থেকে ইনহেরিট করছে।

কোড:

```
class Camera:
    def __init__(self, name):
        self.name = name

    # parent method
    def capture(self):
        print("A photo is captured")

# child classes
class Smart_Phone(Camera):
    def __init__(self, name, resolution):
        super().__init__(name)
        self.resolution = resolution
```

```
    # method overriding
    def capture(self):
        print("Photo is captured by a Smart Phone")

class DSLR(Camera):
    def __init__(self, name, resolution):
        super().__init__(name)
        self.resolution = resolution

    def capture(self):
        print("Photo is captured by a DSLR")

class Drone(Camera):
    def __init__(self, name, resolution):
        super().__init__(name)
        self.resolution = resolution

    def capture(self):
        print("Photo is captured by a Drone")
```

এখন আমরা অবজেক্ট তৈরি করি:

```
phone = Smart_Phone("Phone", 30)
dslr = DSLR("DSLR", 200)
drone = Drone("Drone", 150)

phone.capture()
dslr.capture()
drone.capture()
```

আউটপুট:

```
Photo is captured by a Smart Phone
Photo is captured by a DSLR
Photo is captured by a Drone
```

কী ঘটছে এখানে?

সব ক্লাসেই একই মেথডের নাম capture(),
কিন্তু প্রতিটি ক্লাসে সেই মেথডের আচরণ আলাদা।
এটাই **Polymorphism-**এর আসল শক্তি —একই মেথডের নাম, কিন্তু অবজেক্ট অনুযায়ী আলাদা রূপ।

এখন ধরুন আমরা একটি সাধারণ ফাংশন লিখি, যা যে কোনো ক্যামেরা অবজেক্ট গ্রহণ করবে:

```
def take_photo(camera):
    camera.capture()
```

এখন আমরা একই ফাংশন ব্যবহার করব তিনটি আলাদা অবজেক্টের জন্য:

```
for device in [phone, dslr, drone]:
    take_photo(device)
```

আউটপুট:

```
Photo is captured by a Smart Phone
Photo is captured by a DSLR
Photo is captured by a Drone
```

একই ফাংশন, একই মেথডের নাম — কিন্তু ফলাফল সম্পূর্ণ ভিন্ন।
এটাই Polymorphism-এর সবচেয়ে বড় সুবিধা।

OOP Encapsulation

Encapsulation কী?

Encapsulation মানে হলো **"তথ্যকে সুরক্ষিত রাখা এবং নির্দিষ্ট নিয়ন্ত্রণের মাধ্যমে অ্যাক্সেস করা।"**
অর্থাৎ, অবজেক্টের ডেটা (attributes) এবং কাজ (methods) একসাথে একটি "সংবদ্ধ ইউনিট"-এর মধ্যে রাখা —
এবং বাইরের কেউ যেন সরাসরি সেই ডেটা পরিবর্তন করতে না পারে।
এই ধারণাটি অনেকটা মোবাইল ফোনের ভিতরের সার্কিট বোর্ডের মতো —
আপনি ফোন ব্যবহার করতে পারেন, কিন্তু ভিতরের সার্কিটে সরাসরি হাত দিতে পারেন না।
তেমনই, ক্লাসের অভ্যন্তরীণ ডেটা আমরা **getter** এবং **setter** মেথডের মাধ্যমে নিয়ন্ত্রিতভাবে ব্যবহার করি।

Encapsulation-এর মূল উদ্দেশ্য

- ডেটাকে অনাকাঙ্ক্ষিত পরিবর্তন থেকে রক্ষা করা
- অবজেক্টের অভ্যন্তরীণ বাস্তবায়ন (implementation) গোপন রাখা
- কোডকে আরও নিরাপদ ও নিয়ন্ত্রিত করা

উদাহরণ - Mobile ক্লাস

```
class Mobile:
    def __init__(self, name, model, imei):
        self.__name = name      # private attribute
```

```
        self.__model = model    # private attribute
        self.__imei = imei      # private attribute

    def charge(self):
        print("phone is charging")

    # getter methods
    def imei_getter(self):
        return self.__imei

    def model_getter(self):
        return self.__model

    def name_getter(self):
        return self.__name

    # setter method
    def name_setter(self, name):
        self.__name = name
```

প্রাইভেট অ্যাট্রিবিউট **(__attribute)**

```
self.__imei = imei
```

- দুটি আন্ডারস্কোর (__) দিয়ে শুরু হওয়া অ্যাট্রিবিউটগুলো private বা ব্যক্তিগত হিসেবে বিবেচিত হয়।
- এগুলো ক্লাসের বাইরে থেকে সরাসরি অ্যাক্সেস করা যায় না।
- এর ফলে ডেটা নিরাপদ থাকে, কারণ বাইরের কেউ এটি অনিচ্ছাকৃতভাবে পরিবর্তন করতে পারে না।

উদাহরণস্বরূপ**:**

```
iphone = Mobile("Phone", "17", "1xkaf1")
print(iphone.__imei)  # Error: Attribute not accessible
```

এভাবে সরাসরি প্রাইভেট ডেটা দেখা যাবে না।

Getter Methods

```
def imei_getter(self):
    return self.__imei
```

- Getter মেথড কোনো প্রাইভেট ভ্যারিয়েবলের মান নিয়ন্ত্রিতভাবে পড়ার সুযোগ দেয়।
- এটি একটি **read-only interface** তৈরি করে।

এভাবে আপনি চাইলে বাইরে থেকে IMEI দেখতে পারেন:

```
print(iphone.imei_getter())  # "1xkaf1"
```

Setter Methods

```
def name_setter(self, name):
    self.__name = name
```

- Setter মেথড ডেটা নিয়ন্ত্রিতভাবে পরিবর্তনের সুযোগ দেয়।
- আপনি চাইলে ডেটা পরিবর্তন করতে পারেন, কিন্তু ক্লাসের নির্ধারিত নিয়মের মাধ্যমে।

উদাহরণ:

```
print(iphone.name_getter())   # "Phone"
iphone.name_setter("Phitron")
print(iphone.name_getter())   # "Phitron"
```

ব্যবহার

```
# Registration
iphone = Mobile("Phone", "17", "1xkaf1")

print(iphone.name_getter())  # → Phone

iphone.name_setter("Phitron")  # controlled modification
print(iphone.name_getter())  # → Phitron
```

এখানে ইউজার অবজেক্ট ব্যবহার করছে,
কিন্তু সে জানে না ভিতরে ডেটা কীভাবে রাখা হয়েছে বা পরিবর্তন হচ্ছে —
এই নিয়ন্ত্রণই হলো **Encapsulation-**এর মূল দর্শন।

OOP Abstraction

Abstraction কী?
Abstraction মানে হলো —
“জটিল বাস্তবতা লুকিয়ে রেখে শুধু প্রয়োজনীয় অংশটি ব্যবহারকারীর সামনে উপস্থাপন করা।”
সহজভাবে বললে —
আপনি কাজটা কীভাবে হচ্ছে তা না জেনেই, শুধু কাজটা ব্যবহার করছেন।
উদাহরণ হিসেবে ভাবুন —
আপনি মোবাইল ফোনে “Call” বোতাম চাপেন।
কিন্তু আপনি জানেন না ভিতরে কীভাবে নেটওয়ার্ক সংযোগ, ডায়ালিং সিগন্যাল, বা ট্রান্সমিশন কাজ করছে।
আপনি শুধু “ফোন কল করা” এই কাজটা ব্যবহার করছেন।

এটাই হচ্ছে **Abstraction** — ব্যবহারকারী থেকে অপ্রয়োজনীয় জটিলতা লুকানো।

কেন Abstraction প্রয়োজন?

- কোডকে সহজবোধ্য ও ব্যবহারযোগ্য রাখা
- বাস্তবায়নের জটিলতা লুকিয়ে রাখা
- বিভিন্ন অবজেক্টকে একটি সাধারণ ইন্টারফেসে কাজ করার সুযোগ দেওয়া
- বড় প্রজেক্টে “design contract” বজায় রাখা — অর্থাৎ, কে কোন কাজ করবে তা আগে থেকেই নির্ধারণ করা

উদাহরণ **— Telephone** ক্লাস

```
from abc import ABC, abstractmethod

class Telephone(ABC):

  @abstractmethod
  def make_call(self):
    pass
```

class Telephone(ABC)

ABC মানে Abstract Base Class

এটি এমন একটি ক্লাস, যাকে সরাসরি অবজেক্ট তৈরি করা যায় না।

এটি কেবলমাত্র অন্যান্য ক্লাসের জন্য ব্লুপ্রিন্ট বা নকশা হিসেবে কাজ করে।

@abstractmethod

এটি এমন একটি মেথড, যার কোনো নির্দিষ্ট বাস্তবায়ন নেই।

যেসব ক্লাস Telephone থেকে ইনহেরিট করবে, তাদের এই মেথড অবশ্যই বাস্তবায়ন করতে হবে।

এটি অনেকটা “contract”–এর মতো —

“তুমি আমার সন্তান ক্লাস হলে, তোমাকে make_call() মেথড দিতে হবেই।”

বাস্তব উদাহরণ

```
class Sphone(Telephone):
  def make_call(self):
    print("Making a call using SPhone")

class Iphone(Telephone):
  def make_call(self):
    print("Making a call using IPhone")
```

এখানে দুটি ক্লাস (Sphone, Iphone) Telephone থেকে ইনহেরিট করেছে

এবং নিজেদের মতো করে make_call() মেথড বাস্তবায়ন করেছে।

এভাবে আমরা একই “ফোন কল” ধারণাকে বিভিন্নভাবে বাস্তবায়ন করতে পারি,

যা Abstraction + Polymorphism–এর একটি দুর্দান্ত উদাহরণ।

ব্যবহার

```
ip = Iphone()
ip.make_call()  # Output: Making a call using IPhone
```

এখানে ইউজার শুধু make_call() মেথড কল করছে —
কিন্তু সে জানে না, ভিতরে কীভাবে Telephone ক্লাস কাজ করছে বা “call” কিভাবে হয়।
এই বাস্তবায়ন গোপন রেখে ব্যবহার করা–ই Abstraction।

সারসংক্ষেপ

Abstraction	জটিল বাস্তবায়ন লুকিয়ে রেখে কেবল প্রয়োজনীয় অংশ দেখানো	Telephone
Abstract Class (ABC)	এমন ক্লাস যাকে সরাসরি অবজেক্ট করা যায় না	class Telephone(ABC)
Abstract Method	বাধ্যতামূলকভাবে সাবক্লাসে বাস্তবায়ন করতে হবে এমন মেথড	make_call()
Implementation Class	যে ক্লাসে ওই মেথড বাস্তবায়ন করা হয়	Sphone, Iphone